

科沃斯机器人股份有限公司Ecovacs



科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

Tina Linux 电源管理 开发指南

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本号: 1.2
发布日期: 2023.04.25

版本历史

版本号	日期	制/修订人	内容描述
1.0	2021.11.25	AWA1611	1. 初始版本。 2. 添加 AXP717 说明。
1.1	2022.07.12	AWA1611	1. 添加 Tina 电源管理应用相关说明。 2. 添加 AXP2101 说明。 3. 添加 FAQ 章节。
1.2	2023.04.25	AWA1611	1. 添加 MR527 平台。 2. 添加 Linux-5.15 内核配置说明。

目 录

1 前言	1
1.1 文档简介	1
1.2 目标读者	1
1.3 适用范围	1
2 模块介绍	2
2.1 功能介绍	2
2.2 相关术语介绍	2
2.3 AXP 模块框架介绍	3
2.4 AXP 源码结构介绍	4
2.5 AXP 驱动模块配置介绍	5
2.5.1 Device Tree 配置说明	5
2.5.1.1 AXP717 board.dts 配置	5
2.5.1.2 AXP2101 board.dts 配置	12
2.5.1.3 dts 配置项说明	17
2.5.2 sys_config.fex 配置说明	23
2.5.2.1 AXP717 sys_config.fex 配置	23
2.5.2.2 AXP2101 sys_config.fex 配置	24
2.5.2.3 sys_config 配置项说明	24
2.5.3 uboot-board.dts 配置说明	26
2.5.3.1 AXP2101 uboot-board.dts 配置	26
2.5.3.2 AXP717 uboot-board.dts 配置	28
2.5.3.3 uboot-board.dts 配置项说明	30
2.5.4 kernel menuconfig 配置说明	31
2.5.4.1 AXP2101 配置	32
2.5.4.2 AXP717 配置	36
2.5.4.3 AXP717 配置 on Linux-5.15	37
3 模块使用范例	40
3.1 外部 sysfs 节点	40
3.1.1 Regulator	40
3.1.2 Virtual-consumer	42
3.1.3 Power_supply	43
3.2 Regulator 使用方法	44
3.2.1 内核代码调用 regulator 示例	44
3.2.2 Regulator shell 命令使用示例	45
3.2.3 use_count 查看	46
4 电源管理应用	47

4.1	healthd 机制	48
4.2	充电管理应用	50
4.3	电源管理应用	54
4.3.1	低电量警告	54
4.3.2	低电压关机	55
4.3.3	低电量关机	55
4.3.4	电池过高/过低温关机保护	56
4.3.5	芯片高温警报和关机保护	56
5	调试工具	58
5.1	power key 调试方式	58
5.2	调试节点	58
5.2.1	regulator_summary 节点	58
5.2.2	regmap registers 节点	60
5.2.3	axp_reg 节点	61
5.2.4	debug_mask 节点	61
6	FAQ	62
6.1	如何调整充电电流大小	62
6.2	如何在用户空间获取电池电量等 power_supply 参数	62
6.3	如何查看是否已经在烧录时更新了电池曲线参数	62
6.4	修改开机时 pmu 限制条件	62
6.5	ldo 实际电压与 regulator_summary 的不符	63
6.6	多个模块都请求同一个 regulator 并设置不同的电压，regulator 以哪个模块设置的为准	63

插图

图 2-1	AXP 框架图	3
图 2-2	AXP 软件框架图	3
图 2-3	axp2101-pmu-control-config	32
图 2-4	axp2101-regularot-config	33
图 2-5	axp2101-charger-config	34
图 2-6	axp2101-power-key-config	35
图 2-7	axp2101-virtuaal-config	35
图 2-8	axp717-charger-config	36
图 2-9	axp717-acin-config	37
图 2-10	axp717-config-on-linux-5.15	39
图 4-1	healthd_menuconfig1	48
图 4-2	healthd_menuconfig2	48
图 4-3	healthd_mode_ops	49
图 4-4	is_charger	50
图 4-5	heartbeat	51
图 4-6	process_key	51
图 4-7	power_supply_state	52
图 4-8	update_screen_state	52
图 4-9	charger_flow_chart	53
图 4-10	power_manager_flow_chart	54
图 4-11	low_capacity_warning	55
图 4-12	low_voltage_shutdown	55
图 4-13	low_capacity_shutdown	56
图 4-14	over_ntc_shutdown	56
图 4-15	over_tdie_shutdown	57

1 前言

1.1 文档简介

介绍使用 Tina 系统全志 AXP 平台进行电源管理的使用方法。

1.2 目标读者

适用于开发全志 AXP 平台驱动和电源管理应用功能的开发人员。

1.3 适用范围

表 1-1: 适用 SOC 平台及 AXP 对应表

产品名称	内核版本	AXP 型号
R818	Linux-4.9	AXP717
R853	Linux-4.9	AXP2101
V85X	Linux-4.9	AXP2101
MR527	Linux-5.15	AXP717B
V837S	Linux-4.9	AXP2101
AI985	Linux-5.15	AXP717C

2 模块介绍

2.1 功能介绍

PMU：负责系统各个模块供电、按键开关机、电池充放电管理。

电源管理应用：负责充电管理应用和管理电池异常情况处理。

2.2 相关术语介绍

表 2-1: 术语简介

术语	说明
PMU	电源管理单元，主要包括 regulator、power supply、gpio、power key 这四个子功能部分。
AXP	全志 PMU 平台的系列名称，如 AXP2101、AXP717 等。
LDO	是 low dropout regulator，意为低压差线性稳压器。线性稳压器使用在其线性区域内运行的晶体管或 FET，从应用的输入电压中减去超额的电压，产生经过调节的输出电压。
DC-DC	是直流变直流，即不同直流电源值之间的转换，只要符合这个定义都可以叫 DC-DC 转换器，也包括 LDO。但是一般的说法是把直流变直流由开关方式实现的器件叫 DCDC。
regulator	Linux 内核对 LDO、DC-DC 的管理核心。
USB-Power-Supply	USB 接口对系统的供电。
ACIN-Power-Supply	适配器 ACIN 对系统的供电。
BAT-Power-Supply	电池 BAT 对系统的供电。
Power-Supply	Linux 内核对 USB、ACIN、BAT 供电的管理核心。
MFD	Multi Function Device，Linux 内核对多功能设备 PMU 的管理核心
regmap	Linux 内核用于管理片外模块寄存器的方法。

2.3 AXP 模块框架介绍

AXP 的多功能设备驱动采用 i2c 总线跟主控进行交互，使用 regmap 方式注册访问接口。将 AXP 按照功能抽象出数个设备模块，共同使用父设备 axp mfd 的资源（bus、irq）。基本的软件结构图如下图所示。

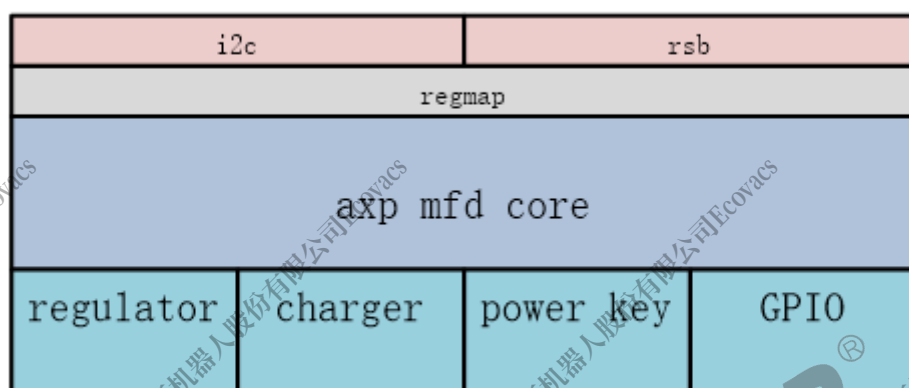


图 2-1: AXP 框架图

将 axp 按照功能划分为几个子设备，分别是 regulator、charger、powe key、gpio。每个子设备作为一个 cell，使用父设备的资源（bus，irq），与不同的内核子系统交互，实现完整的电源管理功能。

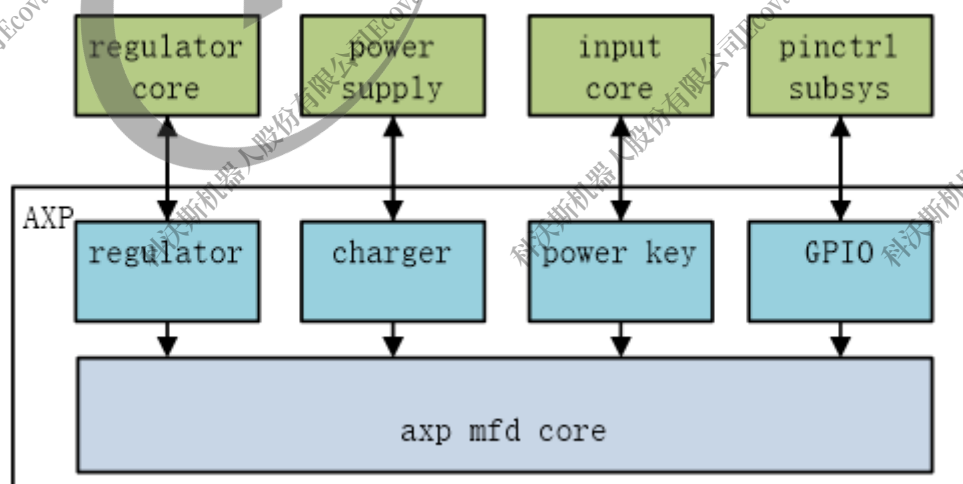


图 2-2: AXP 软件框架图

不同款的 AXP 型号功能框架也会是不一致的，详情各个型号的功能列表如下表所示。

产品名称	regulator	charger	power key	gpio
AXP717&AXP717B	有	有	有	无
AXP2101	有	有	有	无

2.4 AXP 源码结构介绍

- AXP2101

```
${ROOT_DIR}/lichee/{KERNEL_VERSION}/  
drivers/mfd/axp2101.c  
drivers/mfd/axp2101-i2c.c  
drivers/regulator/axp2101-regulator.c  
drivers/input/misc/axp2101-pek.c  
drivers/power/supply/axp2101_battery.c  
drivers/power/supply/axp2101_charger.c  
drivers/power/supply/axp2101_usb_power.c
```

- AXP717

```
${ROOT_DIR}/lichee/{KERNEL_VERSION}/  
drivers/mfd/axp2101.c  
drivers/mfd/axp2101-i2c.c  
drivers/regulator/axp2101-regulator.c  
drivers/input/misc/axp2101-pek.c  
drivers/power/supply/axp2202_battery.c  
drivers/power/supply/axp2202_charger.c  
drivers/power/supply/axp2202_gpio_power.c  
drivers/power/supply/axp2202_usb_power.c
```

- AXP717 on Linux-5.15

```
${ROOT_DIR}/bsp/  
drivers/power/mfd/axp2101.c  
drivers/power/mfd/axp2101-i2c.c  
drivers/power/mfd/pmu-ext-core.c  
drivers/power/mfd/pmu-ext-i2c.c  
drivers/power/regulator/axp2101-regulator.c  
drivers/power/regulator/pmu-ext-regulator.c  
drivers/power/power_key/axp2101-pek.c  
drivers/power/supply/axp2202_battery.c  
drivers/power/supply/axp2202_gpio_power.c  
drivers/power/supply/axp2202_usb_power.c
```

2.5 AXP 驱动模块配置介绍

2.5.1 Device Tree 配置说明

在 Tina 系统中，有两种 dts 文件。一是用于保存芯片所有平台的模块配置 `{CHIP}.dtsi`，二是保存每一个板级平台的设备信息的 `board.dts`。两者的区别主要是：前者主要保存芯片相关的配置，不管芯片外围连接的硬件模块，芯片配置还是保持不变的。而后者是用于保存不同版型之间差异化的配置，跟外围连接的硬件模块是挂钩的。

PMU 模块的 dts 配置是在 `board.dts` 中，`dtsi` 中无用户可用配置。

`board.dts` 路径为：`$(ROOT_DIR)/device/config/chips/${IC}/configs/${TARGET}/board.dts`

说明

`$(ROOT_DIR)`：是 tina SDK 根目录。

`$(IC)`：是芯片型号，如 `r818`，`r853` 等。

`$(TARGET)`：是版级型号，如 `evb1`，`perf1` 等。

技巧

`board.dts` 所在在版级配置目录，tina SDK 环境中，在 `source build/envsetup.sh` 后，可通过 “`cconfigs`” 命令直接跳转过去配置目录，即可找到 `board.dts`。

若方案的内核为 Linux-4.9，则 `board.dts` 就在版本配置目录的上一级，再通过 “`cd ..`” 即可到达 `board.dts` 所在的目录。

PMU 一共包含了 `regulator`，`power supply`，`power key`，PMU 在内核中的设备是多个设备同时存在，并存在层次对应关系。

PMU主设备(MFD)

```
|
+-----> regulator device
|
+-----> power key device
|
+-----> power supply device
|
+-----> wdt device
```

2.5.1.1 AXP717 board.dts 配置

说明

AXP717 在内核中使用的是名为 `axp2202` 的软件框架，因此在 dts 文件中。型号采用软件框架的名字，而不配置为 `axp717`。同理，在 `sysconfig.fex` 和 `kernel menuconfig` 中也一样。

```
pmu0: pmu@34 {
    compatible = "x-powers,axp2202";
    reg = <0x34>;
    interrupts = <0 IRQ_TYPE_LEVEL_LOW>;
    interrupt-parent = <&nmi_intc>;
    x-powers,drive-vbus-en;
    pmu_reset = <0>;
```

```
pmu_irq_wakeup = <1>;
pmu_hot_shutdown = <1>;

wakeup-source;

//interrupt-controller;
//#interrupt-cells = <1>;

usb_power_supply: usb_power_supply {
    compatible = "x-powers,axp2202-usb-power-supply";

    pmu_usbpc_vol = <4600>;
    pmu_usbpc_cur = <500>;
    pmu_usbad_vol = <4000>;
    pmu_usbad_cur = <2500>;

    pmu_boost_vol = <5126>;
    pmu_bc12_en;
    pmu_cc_logic_en = <1>;
    /* pmu_boost_en; */

    pmu_usb_typec_used = <1>;
    wakeup_usb_in;
    wakeup_usb_out;

    status = "okay";
};

/* cvin */
gpio_power_supply: gpio_power_supply {
    compatible = "x-powers,gpio-supply";
    status = "disabled";

    wakeup_gpio;
};

bat_power_supply: bat-power-supply {
    compatible = "x-powers,axp2202-bat-power-supply";
    param = <&axp2202_parameter>;
    status = "okay";

    pmu_chg_ic_temp = <1>;

    pmu_battery_rdc = <147>;
    pmu_battery_cap = <1771>;
    pmu_runtime_chgcur = <1000>;
    pmu_suspend_chgcur = <1500>;
    pmu_shutdown_chgcur = <1500>;
    pmu_terminal_chgcur = <128>;
    pmu_init_chgvol = <4200>;
    pmu_battery_warning_level1 = <15>;
    pmu_battery_warning_level2 = <0>;
    pmu_chgled_func = <0>;
    pmu_chgled_type = <0>;
    pmu_bat_para1 = <0>;
    pmu_bat_para2 = <0>;
    pmu_bat_para3 = <0>;
    pmu_bat_para4 = <0>;
    pmu_bat_para5 = <0>;
    pmu_bat_para6 = <0>;
    pmu_bat_para7 = <2>;
```

```
pmu_bat_para8 = <3>;
pmu_bat_para9 = <4>;
pmu_bat_para10 = <6>;
pmu_bat_para11 = <9>;
pmu_bat_para12 = <14>;
pmu_bat_para13 = <26>;
pmu_bat_para14 = <38>;
pmu_bat_para15 = <49>;
pmu_bat_para16 = <52>;
pmu_bat_para17 = <56>;
pmu_bat_para18 = <60>;
pmu_bat_para19 = <64>;
pmu_bat_para20 = <70>;
pmu_bat_para21 = <77>;
pmu_bat_para22 = <83>;
pmu_bat_para23 = <87>;
pmu_bat_para24 = <90>;
pmu_bat_para25 = <95>;
pmu_bat_para26 = <99>;
pmu_bat_para27 = <99>;
pmu_bat_para28 = <100>;
pmu_bat_para29 = <100>;
pmu_bat_para30 = <100>;
pmu_bat_para31 = <100>;
pmu_bat_para32 = <100>;
pmu_bat_temp_enable = <0>;
pmu_bat_charge_ltf = <1105>;
pmu_bat_charge_hrf = <121>;
pmu_bat_shutdown_ltf = <1381>;
pmu_bat_shutdown_hrf = <89>;
pmu_bat_temp_para1 = <2814>;
pmu_bat_temp_para2 = <2202>;
pmu_bat_temp_para3 = <1737>;
pmu_bat_temp_para4 = <1381>;
pmu_bat_temp_para5 = <1105>;
pmu_bat_temp_para6 = <890>;
pmu_bat_temp_para7 = <722>;
pmu_bat_temp_para8 = <484>;
pmu_bat_temp_para9 = <332>;
pmu_bat_temp_para10 = <233>;
pmu_bat_temp_para11 = <196>;
pmu_bat_temp_para12 = <166>;
pmu_bat_temp_para13 = <141>;
pmu_bat_temp_para14 = <121>;
pmu_bat_temp_para15 = <89>;
pmu_bat_temp_para16 = <66>;

wakeup_bat_out;
/* wakeup_bat_in; */
/* wakeup_bat_charging; */
/* wakeup_bat_charge_over; */
/* wakeup_low_warning1; */
/* wakeup_low_warning2; */
/* wakeup_bat_unttemp_work; */
/* wakeup_bat_ovtemp_work; */
/* wakeup_bat_unttemp_chg; */
/* wakeup_bat_ovtemp_chg; */
};

powerkey0: powerkey@0 {
```

```
compatible = "x-powers,axp2101-pek";
pmu_powkey_off_time = <6000>;
pmu_powkey_off_func = <0>;
pmu_powkey_off_en = <1>;
pmu_powkey_long_time = <1500>;
pmu_powkey_on_time = <512>;
wakeup_rising;
wakeup_falling;
status = "okay";
};

regulator0: regulators@0 {
    reg_dcdc1: dcdc1 {
        regulator-name = "axp2202-dcdc1";
        regulator-min-microvolt = <500000>;
        regulator-max-microvolt = <1540000>;
        regulator-ramp-delay = <2500>;
        regulator-enable-ramp-delay = <1000>;
        regulator-boot-on;
        regulator-always-on;
    };
    reg_dcdc2: dcdc2 {
        regulator-name = "axp2202-dcdc2";
        regulator-min-microvolt = <500000>;
        regulator-max-microvolt = <3400000>;
        regulator-ramp-delay = <2500>;
        regulator-enable-ramp-delay = <1000>;
        regulator-boot-on;
        regulator-always-on;
    };
    reg_dcdc3: dcdc3 {
        regulator-name = "axp2202-dcdc3";
        regulator-min-microvolt = <500000>;
        regulator-max-microvolt = <1840000>;
        regulator-ramp-delay = <2500>;
        regulator-enable-ramp-delay = <1000>;
        regulator-always-on;
    };
    reg_dcdc4: dcdc4 {
        regulator-name = "axp2202-dcdc4";
        regulator-min-microvolt = <1000000>;
        regulator-max-microvolt = <3700000>;
        regulator-ramp-delay = <2500>;
        regulator-enable-ramp-delay = <1000>;
    };
    reg_rtcldo: rtcldo {
        /* RTC_LDO is a fixed, always-on regulator */
        regulator-name = "axp2202-rtcldo";
        regulator-min-microvolt = <1800000>;
        regulator-max-microvolt = <1800000>;
        regulator-boot-on;
        regulator-always-on;
    };
    reg_aldo1: aldo1 {
        regulator-name = "axp2202-aldo1";
        regulator-min-microvolt = <500000>;
        regulator-max-microvolt = <3500000>;
        regulator-enable-ramp-delay = <1000>;
    };
    reg_aldo2: aldo2 {
```

```
regulator-name = "axp2202-aldo2";
regulator-min-microvolt = <500000>;
regulator-max-microvolt = <3500000>;
regulator-enable-ramp-delay = <1000>;
};
reg_aldo3: aldo3 {
    regulator-name = "axp2202-aldo3";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
    regulator-always-on;
    regulator-boot-on;
};
reg_aldo4: aldo4 {
    regulator-name = "axp2202-aldo4";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
    regulator-always-on;
    regulator-boot-on;
};
reg_bldo1: bldo1 {
    regulator-name = "axp2202-bldo1";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_bldo2: bldo2 {
    regulator-name = "axp2202-bldo2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_bldo3: bldo3 {
    regulator-name = "axp2202-bldo3";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_bldo4: bldo4 {
    regulator-name = "axp2202-bldo4";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_cldo1: cldo1 {
    regulator-name = "axp2202-cldo1";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_cldo2: cldo2 {
    regulator-name = "axp2202-cldo2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_cldo3: cldo3 {
```

```
regulator-name = "axp2202-cldo3";
regulator-min-microvolt = <500000>;
regulator-max-microvolt = <3500000>;
regulator-ramp-delay = <2500>;
regulator-enable-ramp-delay = <1000>;
regulator-boot-on;
};
reg_cldo4: cldo4 {
    regulator-name = "axp2202-cldo4";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-enable-ramp-delay = <1000>;
};
reg_cpusldo: cpusldo {
    /* cpus */
    regulator-name = "axp2202-cpusldo";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <1400000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_drivevbus: drivevbus {
    regulator-name = "axp2202-drivevbus";
    regulator-enable-ramp-delay = <1000>;
};

};

virtual-dcdc1 {
    compatible = "xpower-vregulator,dcdc1";
    dcdc1-supply = <&reg_dcdc1>;
};
virtual-dcdc2 {
    compatible = "xpower-vregulator,dcdc2";
    dcdc2-supply = <&reg_dcdc2>;
};
virtual-dcdc3 {
    compatible = "xpower-vregulator,dcdc3";
    dcdc3-supply = <&reg_dcdc3>;
};
virtual-dcdc4 {
    compatible = "xpower-vregulator,dcdc4";
    dcdc4-supply = <&reg_dcdc4>;
};
virtual-rtcldo {
    compatible = "xpower-vregulator,rtcldo";
    rtcldo-supply = <&reg_rtcldo>;
};
virtual-aldo1 {
    compatible = "xpower-vregulator,aldo1";
    aldo1-supply = <&reg_aldo1>;
};
virtual-aldo2 {
    compatible = "xpower-vregulator,aldo2";
    aldo2-supply = <&reg_aldo2>;
};
virtual-aldo3 {
    compatible = "xpower-vregulator,aldo3";
    aldo3-supply = <&reg_aldo3>;
};
```

```
virtual-ald04 {
    compatible = "xpower-vregulator,ald04";
    ald04-supply = <&reg_ald04>;
};
virtual-bl0d01 {
    compatible = "xpower-vregulator,bl0d01";
    bl0d01-supply = <&reg_bl0d01>;
};
virtual-bl0d02 {
    compatible = "xpower-vregulator,bl0d02";
    bl0d02-supply = <&reg_bl0d02>;
};
virtual-bl0d03 {
    compatible = "xpower-vregulator,bl0d03";
    bl0d03-supply = <&reg_bl0d03>;
};
virtual-bl0d04 {
    compatible = "xpower-vregulator,bl0d04";
    bl0d04-supply = <&reg_bl0d04>;
};
virtual-cl0d01 {
    compatible = "xpower-vregulator,cl0d01";
    cl0d01-supply = <&reg_cl0d01>;
};
virtual-cl0d02 {
    compatible = "xpower-vregulator,cl0d02";
    cl0d02-supply = <&reg_cl0d02>;
};
virtual-cl0d03 {
    compatible = "xpower-vregulator,cl0d03";
    cl0d03-supply = <&reg_cl0d03>;
};
virtual-cl0d04 {
    compatible = "xpower-vregulator,cl0d04";
    cl0d04-supply = <&reg_cl0d04>;
};
virtual-cpusl0d0 {
    compatible = "xpower-vregulator,cpusl0d0";
    cpusl0d0-supply = <&reg_cpusl0d0>;
};
virtual-drivevbus {
    compatible = "xpower-vregulator,drivevbus";
    drivevbus-supply = <&reg_drivevbus>;
};
axp_gpio0: axp_gpio@0 {
    gpio-controller;
    #size-cells = <0>;
    #gpio-cells = <6>;
    status = "okay";
};

/ {
    axp2202_parameter:axp2202-parameter {
        select = "battery-model";
        battery-model {
            parameter = /bits/ 8 <0x01 0xf5 0x40 0x00 0x1b 0x1e 0x28 0x0f
0x0c 0x1e 0x32 0x02 0x14 0x05 0x0a 0x04
0x74 0xfb 0xc8 0x0d 0x43 0x10 0x36 0xfb
0x46 0x01 0xea 0x0d 0x2a 0x06 0x36 0x05>;
        };
    };
};
```

```

0xf4 0x0a 0xb5 0x0f 0x42 0x0e 0xe6 0x09
0x9a 0x0e 0x42 0x0e 0x3b 0x04 0x2d 0x04
0x23 0x09 0x18 0x0e 0x09 0x0e 0x04 0x08
0xf7 0x0d 0xda 0x0d 0xd0 0x03 0xbb 0x03
0x9d 0x08 0x7f 0x0d 0x6a 0x0d 0x55 0x07
0xc2 0x57 0x2b 0x27 0x1e 0x0d 0x14 0x08
0xc5 0x98 0x7e 0x66 0x4e 0x44 0x38 0x1a
0x12 0x0a 0xf6 0x00 0x00 0xf6 0x00 0xf6
0x00 0xfb 0x00 0x00 0xfb 0x00 0x00 0xfb
0x00 0x00 0xf6 0x00 0x00 0xf6 0x00 0xf6
0x00 0xfb 0x00 0x00 0xfb 0x00 0x00 0xfb
0x00 0x00 0xf6 0x00 0x00 0xf6 0x00 0xf6>;
};
};
};

```

2.5.1.2 AXP2101 board.dts 配置

```

pmu0: pmu@0 {
    compatible = "x-powers,axp2101";
    reg = <0x34>;
    #address-cells = <1>;
    #size-cells = <0>;
    interrupts = <0 IRQ_TYPE_LEVEL_LOW>;
    interrupt-parent = <&nmi_intc>;
    pmu_hot_shutdown = <1>;
    pmu_hot_shutdown_value = <125>;
    pmu_over_current = <1>;
    status = "okay";
    wakeup-source;

    usb_power_supply: usb_power_supply {
        compatible = "x-powers,axp2101-usb-power-supply";
        status = "okay";

        pmu_usbpc_vol = <4600>;
        pmu_usbpc_cur = <500>;
        pmu_usbad_vol = <4000>;
        pmu_usbad_cur = <2500>;

        wakeup_usb_in;
        wakeup_usb_out;
    };

    bat_power_supply: bat-power-supply {
        compatible = "x-powers,axp2101-bat-power-supply";
        param = <&axp2101_parameter>;
        status = "okay";

        pmu_ityrm_limit = <125>;
        pmu_pre_chg = <100>;
        pmu_chg_ic_temp = <0>;
        pmu_battery_cap = <888>;
        pmu_runtime_chgcur = <500>;
        pmu_suspend_chgcur = <1000>;
        pmu_shutdown_chgcur = <1000>;
        pmu_init_chgvol = <4200>;
    };
}

```

```
pmu_battery_warning_level1 = <15>;  
pmu_battery_warning_level2 = <5>;  
pmu_chgled_type = <0x0>;
```

```
pmu_bat_temp_enable = <1>;  
pmu_bat_ts_current = <50>;  
pmu_jetia_en = <1>;  
pmu_bat_charge_ltf = <1350>;  
pmu_bat_charge_hft = <150>;  
pmu_bat_shutdown_ltf = <2100>;  
pmu_bat_shutdown_hft = <111>;  
pmu_jetia_cool = <725>;  
pmu_jetia_warm = <290>;  
pmu_jcool_ifall = <1>;  
pmu_jwarm_ifall = <1>;  
pmu_bat_temp_para1 = <4350>;  
pmu_bat_temp_para2 = <2650>;  
pmu_bat_temp_para3 = <2100>;  
pmu_bat_temp_para4 = <1650>;  
pmu_bat_temp_para5 = <1350>;  
pmu_bat_temp_para6 = <1100>;  
pmu_bat_temp_para7 = <850>;  
pmu_bat_temp_para8 = <600>;  
pmu_bat_temp_para9 = <456>;  
pmu_bat_temp_para10 = <290>;  
pmu_bat_temp_para11 = <245>;  
pmu_bat_temp_para12 = <208>;  
pmu_bat_temp_para13 = <175>;  
pmu_bat_temp_para14 = <150>;  
pmu_bat_temp_para15 = <111>;  
pmu_bat_temp_para16 = <83>;
```

```
wakeup_bat_out;  
/* wakeup_bat_in; */  
/* wakeup_bat_charging; */  
/* wakeup_bat_charge_over; */  
/* wakeup_low_warning1; */  
/* wakeup_low_warning2; */  
/* wakeup_bat_unttemp_work; */  
/* wakeup_bat_ovtemp_work; */  
/* wakeup_bat_unttemp_chg; */  
/* wakeup_bat_ovtemp_chg; */  
};
```

```
powerkey0: powerkey@0 {  
    status = "okay";  
    compatible = "x-powers,axp2101-pek";  
    pmu_powkey_off_time = <6000>;  
    pmu_powkey_off_func = <0>;  
    pmu_powkey_off_en = <1>;  
    pmu_powkey_long_time = <1500>;  
    pmu_powkey_on_time = <512>;  
    wakeup_rising;  
    wakeup_falling;  
};
```

```
regulator0: regulators@0 {  
    reg_dcdc1: dcdc1 {
```

```
regulator-name = "axp2101-dcdc1";
regulator-min-microvolt = <1500000>;
regulator-max-microvolt = <3400000>;
regulator-boot-on;
regulator-always-on;
};
reg_dcdc2: dcdc2 {
    regulator-name = "axp2101-dcdc2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <1540000>;
};
reg_dcdc3: dcdc3 {
    regulator-name = "axp2101-dcdc3";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3400000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_dcdc4: dcdc4 {
    regulator-name = "axp2101-dcdc4";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <1840000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_dcdc5: dcdc5 {
    regulator-name = "axp2101-dcdc5";
    regulator-min-microvolt = <1400000>;
    regulator-max-microvolt = <3700000>;
};
reg_rtcldo: rtcldo {
    /* RTC_LDO is a fixed, always-on regulator */
    regulator-name = "axp2101-rtcldo";
    regulator-min-microvolt = <1800000>;
    regulator-max-microvolt = <1800000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_rtcldo1: rtcldo1 {
    regulator-name = "axp2101-rtcldo1";
    regulator-min-microvolt = <1800000>;
    regulator-max-microvolt = <1800000>;
};
reg_aldo1: aldo1 {
    regulator-name = "axp2101-aldo1";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_aldo2: aldo2 {
    regulator-name = "axp2101-aldo2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
};
reg_aldo3: aldo3 {
    regulator-name = "axp2101-aldo3";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
};
```

```
reg_aldo4: aldo4 {
    regulator-name = "axp2101-aldo4";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
};
reg_bldo1: bldo1 {
    regulator-name = "axp2101-bldo1";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
    regulator-boot-on;
    regulator-always-on;
};
reg_bldo2: bldo2 {
    regulator-name = "axp2101-bldo2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
};
reg_dldo1: dldo1 {
    regulator-name = "axp2101-dldo1";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <3500000>;
};
reg_dldo2: dldo2 {
    regulator-name = "axp2101-dldo2";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <1400000>;
};
reg_cpusldo: cpusldo {
    regulator-name = "axp2101-cpusldo";
    regulator-min-microvolt = <500000>;
    regulator-max-microvolt = <1400000>;
};

};
virtual-dcdc1 {
    compatible = "xpower-vregulator,dcdc1";
    dcdc1-supply = <&reg_dcdc1>;
};
virtual-dcdc2 {
    compatible = "xpower-vregulator,dcdc2";
    dcdc2-supply = <&reg_dcdc2>;
};
virtual-dcdc3 {
    compatible = "xpower-vregulator,dcdc3";
    dcdc3-supply = <&reg_dcdc3>;
};

virtual-dcdc4 {
    compatible = "xpower-vregulator,dcdc4";
    dcdc4-supply = <&reg_dcdc4>;
};
virtual-dcdc5 {
    compatible = "xpower-vregulator,dcdc5";
    dcdc5-supply = <&reg_dcdc5>;
};

virtual-aldo1 {
    compatible = "xpower-vregulator,aldo1";
    aldo1-supply = <&reg_aldo1>;
```

```

};
virtual-aldo2 {
    compatible = "xpower-vregulator,aldo2";
    aldo2-supply = <&reg_aldo2>;
};
virtual-aldo3 {
    compatible = "xpower-vregulator,aldo3";
    aldo3-supply = <&reg_aldo3>;
};
virtual-aldo4 {
    compatible = "xpower-vregulator,aldo4";
    aldo4-supply = <&reg_aldo4>;
};

virtual-bldo1 {
    compatible = "xpower-vregulator,bldo1";
    bldo1-supply = <&reg_bldo1>;
};
virtual-bldo2 {
    compatible = "xpower-vregulator,bldo2";
    bldo2-supply = <&reg_bldo2>;
};

virtual-dldo1 {
    compatible = "xpower-vregulator,dldo1";
    dldo1-supply = <&reg_dldo1>;
};
virtual-dldo2 {
    compatible = "xpower-vregulator,dldo2";
    dldo2-supply = <&reg_dldo2>;
};

axp_gpio0: axp_gpio@0 {
    gpio-controller;
    #size-cells = <0>;
    #gpio-cells = <6>;
    status = "okay";
};

/*
axp2101_parameter:axp2101-parameter {
    select = "battery-model";

    battery-model {
        parameter = /bits/ 8 <0x01 0xF5 0x00 0x00 0xFB 0x00 0x00 0xFB
        0x00 0x1E 0x32 0x01 0x14 0x04 0xD8 0x04
        0x74 0xFD 0x58 0x0B 0xB3 0x10 0x3F 0xFB
        0xC8 0x00 0xBE 0x03 0x4E 0x06 0x3F 0x06
        0x02 0x0A 0xD3 0x0F 0x74 0x0F 0x31 0x09
        0xE5 0x0E 0xB9 0x0E 0xC0 0x04 0xBE 0x04
        0xBB 0x09 0xB4 0x0E 0xA0 0x0E 0x92 0x09
        0x79 0x0E 0x4C 0x0E 0x27 0x03 0xFC 0x03
        0xD5 0x08 0xBC 0x0D 0x9C 0x0D 0x55 0x06
        0xB8 0x2E 0x24 0x2E 0x2E 0x24 0x2E 0x24
        0xC5 0x98 0x7E 0x66 0x4E 0x44 0x38 0x1A
        0x12 0x0A 0xF6 0x00 0x00 0xF6 0x00 0xF6
        0x00 0xFB 0x00 0x00 0xFB 0x00 0x00 0xFB
        0x00 0x00 0xF6 0x00 0x00 0xF6 0x00 0xF6
        0x00 0xFB 0x00 0x00 0xFB 0x00 0x00 0xFB
    }
}

```

```
0x00 0x00 0xF6 0x00 0x00 0xF6 0x00 0xF6>;  
};  
};  
};
```

2.5.1.3 dts 配置项说明

说明

PMU 为 I2C 设备，PMU 设备配置需要写在 i2c 节点内。

• PMU 属性配置

```
reg <u32>  
    i2c寄存器地址  
  
interrupts <args>  
    中断配置，参考内核中断配置文档  
  
interrupt-parent <phandle>  
    上级中断控制器结点  
  
wakeup-source <bool>  
    是否作为唤醒源  
    0:disable  
    1:enable  
  
x-powers,drive-vbus-en <bool>  
    set N_VBUSEN pin as an output pin to control an external regulator to drive VBus  
  
pmu_reset <bool>  
    when power key press longer than 16s, PMU reset or not.  
    0: not reset  
    1: reset  
  
pmu_irq_wakeup  
    press irq wakeup or not when sleep or power down.  
    0: not wakeup  
    1: wakeup  
  
pmu_hot_shutdown  
    when PMU over temperature protect or not.  
    0: disable  
    1: enable
```

• power_supply 配置

power_supply 属性配置，包括 usb_power_supply、gpio_power_supply 和 battery_power_supply。

对于 usb_power_supply 属性配置如下：

pmu_usbpc_vol <u32>
usb pc输入电压限制值，单位为mV

pmu_usbpc_cur <u32>
usb pc输入电流限制值，单位为mA

pmu_usbad_vol <u32>
usb adaptor输入电压限制值(vimdpm)，单位为mV

pmu_usbad_cur <u32>
usb adaptor输入电流限制值，单位为mA

pmu_boost_vol <u32>
打开boost给usb口供电的电压值，单位为mV

pmu_bc12_en <bool>
是否打开BC1.2协议功能

pmu_cc_logic_en <bool>
是否使能cc协议功能

pmu_boost_en <bool>
是否在初始化时打开boost功能

pmu_usb_typec_used <bool>
usb接口为type-c，使用type-c模式

wakeup_usb_in <bool>
usb插入唤醒使能

wakeup_usb_out <bool>
usb拔出唤醒使能

📖 说明

在使用 type-c 时，还需将 usb 驱动中的 “usb_detect_type” 节点同步配置成 2，才能使用 typec 识别 usb 设备的功能。

对于 gpio_power_supply：是控制 ac in 的配置，在 AXP717 和 AXP2101 上默认不使用。

对于 battery_power_supply 属性配置如下：

param <phandler>
指定电池充电结点
电池参数由多个字节组成，在param指定的phandler结点里面添加电池结点，然后通过select选择参数结点名字用来指定哪个结点。
参考实例：
根据前面的axp2202-parameter结点。
电池参数根据使用的电池不同，通过仪器测量出来。
视具体的board.dts属性节点而定，部分型号PMU驱动暂未支持该功能

pmu_chg_ic_temp <u32>
1: TS current source always on
0: TS current source off

pmu_battery_rdc <u32>
电池内阻，单位为mΩ

pmu_battery_cap <u32>
电池容量，单位为mAh

pmu_runtime_chgcur <u32>
运行时constant充电电流限制，单位为mA

pmu_suspend_chgcur <u32>
休眠时constant充电电流限制，单位为mA

pmu_shutdown_chgcur <u32>
关机时constant充电电流限制，单位为mA

pmu_terminal_chgcur <u32>
截止电流，停止充电的标志位之一，单位为mA

pmu_init_chgvol <u32>
电池满充电压，单位为mV

pmu_battery_warning_level1 <u32>
5-20 5% - 20% warning level1
电池低电量警告，当芯片检测电池电量从高到低跌到了设置的level1的值，capacity < warning_level1，就会触发warning_level1中断，从而再进行对应操作。
如果capacity ≥ warning_level1则会清掉该中断。
如AXP717，当触发warning1中断时，默认是不发生操作，需要修改对应代码进行定制化操作。

pmu_battery_warning_level2 <u32>
0-15 0% - 15% warning level2
意义同level1，当电池电量从从高到低跌落，capacity < warning_level2，就会触发warning_level2中断。
当电池电量 warning_level2 ≤ capacity < warning_level1 则会清掉 level2中断。

pmu_chgled_func <u32>;
CHGLED pin control
0: controlled by pmu
1: controlled by Charger

pmu_chgled_type <u32>
CHGLED Type select when pmu_chgled_func is 0
0: display with type A function
1: display with type B function
3: output controlled by the register of chgled_out_ctrl

pmu_bat_para1 <u32>
pmu_bat_para2 <u32>
...
pmu_bat_para32 <u32>
电池曲线参数
电池参数根据使用的电池不同，通过仪器测量出来

pmu_bat_temp_enable <u32>
设置电池温度检测、ntc是否使能

pmu_bat_charge_ltf <u32>
触发电池低温停充的TS pin电压阈值，单位：mV
默认：1105mV
范围：0-8160mV

pmu_bat_charge_hrf <u32>
触发电池高温停充的TS pin电压阈值，单位：mV
默认：121mV
范围：0-510mV

pmu_bat_shutdown_ltf <u32>
非充电模式下，触发电池低温中断的TS pin电压阈值，单位：mV

默认：1381mV

pmu_bat_shutdown_hf <u32>
默认：89mV
范围：0-510mV

pmu_bat_temp_para1 <u32>
电池包-25度对应的TS pin电压，单位：mV

pmu_bat_temp_para2 <u32>
电池包-15度对应的TS pin电压，单位：mV

pmu_bat_temp_para3 <u32>
电池包-10度对应的TS pin电压，单位：mV

pmu_bat_temp_para4 <u32>
电池包-5度对应的TS pin电压，单位：mV

pmu_bat_temp_para5 <u32>
电池包0度对应的TS pin电压，单位：mV

pmu_bat_temp_para6 <u32>
电池包5度对应的TS pin电压，单位：mV

pmu_bat_temp_para7 <u32>
电池包10度对应的TS pin电压，单位：mV

pmu_bat_temp_para8 <u32>
电池包20度对应的TS pin电压，单位：mV

pmu_bat_temp_para9 <u32>
电池包30度对应的TS pin电压，单位：mV

pmu_bat_temp_para10 <u32>
电池包40度对应的TS pin电压，单位：mV

pmu_bat_temp_para11 <u32>
电池包45度对应的TS pin电压，单位：mV

pmu_bat_temp_para12 <u32>
电池包50度对应的TS pin电压，单位：mV

pmu_bat_temp_para13 <u32>
电池包55度对应的TS pin电压，单位：mV

pmu_bat_temp_para14 <u32>
电池包60度对应的TS pin电压，单位：mV

pmu_bat_temp_para15 <u32>
电池包70度对应的TS pin电压，单位：mV

pmu_bat_temp_para16 <u32>
电池包80度对应的TS pin电压，单位：mV
不同电池包的温敏电阻特性不一样，根据电池包的TS温敏电阻手册，找到pmu_bat_temp_para[1-16]对应温度点的电阻阻值，将阻值除以20得到的电压数值（单位：mV），将电压数值填进pmu_bat_temp_para[1-16]的节点中即可

wakeup_bat_out <bool>
电池拔出唤醒使能

wakeup_bat_charging <bool>

电池充电唤醒使能

wakeup_bat_charge_over <bool>
电池充电结束唤醒使能

wakeup_low_warning1 <bool>
电池低电量告警唤醒使能

wakeup_low_warning2 <bool>
电池低电量告警2唤醒使能

wakeup_bat_untemp_chg <bool>
电池低温充电唤醒使能

wakeup_bat_ovtemp_chg <bool>
电池超温充电唤醒使能

wakeup_bat_untemp_work <bool>
电池低温工作唤醒使能

wakeup_bat_ovtemp_work <bool>
电池高温工作唤醒使能

• power key 属性配置

power key 设备为电源按键设备。power key 属性配置如下：

pmu_powkey_off_time <u32>

控制按下多长时间响应poweroff事件

可选的值为：

4000 4s
6000 6s
8000 8s
10000 10s

pmu_powkey_off_func <u32>

控制power_off事件功能,如果不配置，默认为power-off

0: power_off
1: 复位系统

pmu_powkey_off_en <bool>

控制按键关机使能

1: PWRON > OFFLEVEL AS poweroff source enable
0: PWRON > OFFLEVEL as poweroff source disable

pmu_powkey_long_time <u32>

控制ponlevel 寄存器0x27[5:4]

1000 1s
1500 1.5s
2000 2s
2500 2.5s

pmu_powkey_on_time <u32>

控制按钮按下多长时间开机

128 0.128s
512 0.512s
1000 1s
2000 2s

wakeup_rising <bool>
控制是否弹起按钮唤醒系统

wakeup_falling <bool>
控制是否按下按钮唤醒系统

说明

X-POWERS 电池参数更新说明：

axp717 使用 axp2202_parameter 下的 128 个参数来设置电池充电属性、关机电压和电池容量等电池属性。

若要更新该参数，可参考 1 号通上的文档《X-POWERS 电池参数测试系统使用说明_V1.4.pdf》来进行电池参数测试。

支持该参数的 AXP 可如下表所示。

表 2-3: 支持 axp-parameter 的 AXP 型号列表

AXP 型号
AXP717
AXP2101

- regulator 属性配置

regulator 为系统 regulator_dev 设备，每个 regulator_dev 代表一路电源，设备通过对 regulator_dev 的引用建立 regulator，用来实现对电源的电压设置等功能。regulator 属性配置，参考内核原生 regulator 使用文档：Documentation/devicetree/bindings/regulator/regulator.txt。

regulator 配置如下：

```
reg_aldo1: aldo1{
    regulator-name = "axp2101-dcdc1";
    为电源设备的名称

    regulator-min-microvolt = <1500000>;
    电源的最小值，单位：uV

    regulator-max-microvolt = <3400000>;
    电源的最大值，单位：uV

    regulator-ramp-delay = <2500>;
    电源的调压延时，单位：us

    regulator-enable-ramp-delay = <1000>;
    电源从关闭到开启的使能延时，单位：us

    regulator-boot-on;
    电源从启动时开启，在内核启动时，就按照dts配置加载了该路regulator。同时，进入系统后通过软件读取也能获知该路
    regulator被加载了。

    regulator-always-on;
    电源保持常开，不会由于调用regulator等API接口而关闭。
};
```

2.5.2 sys_config.fex 配置说明

在 sysconfig 中定义了 PMU 的 regulator 输出信息及板型 PMU 类型，在 boot0 和 uboot 会通过解析这部分属性来执行调压等操作。

2.5.2.1 AXP717 sys_config.fex 配置

```

;-----
;[target] system bootup configuration
;boot_clock   = CPU boot frequency, Unit: MHz
;storage_type  = boot medium, 0-nand, 1-card0, 2-card2, -1(default)auto scan
;advert_enable = 0-close advert logo  1-open advert logo  (只有多核启动下有效)
;power_mode    = axp_type, 0:axp81X, 1:dummy, 2:axp806, 3:axp2202, 4:axp858
;-----
[target]
boot_clock   = 1008
storage_type  = -1
advert_enable = 0
burn_key      = 1
dragonboard_test= 0
power_mode    = 3
;-----
; system configuration
; ?
;dcdc1_vol      ---set dcdc1 voltage,mV,500-1200,10mV/step
;               1220-3400,20mV/step
;dcdc2_vol      ---set dcdc2 voltage,mV,500-1200,10mV/step
;               1220-1540,20mV/step
;aldo1_vol      ---set aldo1 voltage,mV,500-3500,100mV/step
;dldo1_vol      ---set dldo1 voltage,mV,500-3500,100mV/step
;-----
[power_sply]
dcdc3_vol      = 1001200
aldo3_vol      = 1003300
aldo4_vol      = 1001800
blldo2_vol     = 1002500
clldo1_vol     = 1001800
clldo3_vol     = 1003300
cpusldo_vol    = 1009000
dcdc1_mode     = 1
dcdc2_mode     = 1
;-----
; gpio_bias
; set gpio group withstand voltage
; pc_bias = 1800 is emmc
; pc_bias = 3300 is nand
;-----
[gpio_bias]
device_type    = "gpio_bias"
pl_bias        = 3300
pl_supply      = "aldo3_vol"
pc_bias        = 1800
pc_supply      = "clldo1_vol"

```

```
;pc_bias          = 3300
;pc_supply        = "cldo3_vol"

[power_delay]
device_type       = "power_delay"
aldo3_vol_delay   = 20000
```

2.5.2.2 AXP2101 sys_config.fex 配置

```
;-----
; system configuration
; ?
;dcdc1_vol        ---set dcdc1 voltage,mV,1600-3400,100mV/step
;dcdc2_vol        ---set dcdc2 voltage,mV,600-1540,20mV/step
;dcdc3_vol        ---set dcdc3 voltage,mV,600-1860,20mV/step
;dcdc4_vol        ---set dcdc4 voltage,mV,600-1540,20mV/step
;dcdc5_vol        ---set dcdc5 voltage,mV,1000-2550,50mV/step
;aldo2_vol        ---set aldo2 voltage,mV,700-3300,100mV/step
;aldo3_vol        ---set aldo3 voltage,mV,700-3300,100mV/step
;-----
[power_sply]
dcdc1_vol         = 3000
dcdc2_vol         = 1200
dcdc3_vol         = 1200
dcdc4_vol         = 1200
dcdc5_vol         = 1500
aldo2_vol         = 1800
aldo3_vol         = 3000
;-----
; startup configuration
; ?
;pmu_safe_vol      startup vol
;pmu_safe_ratio    startup ratio
;-----
[charger0]
pmu_safe_vol      = 3400
pmu_safe_ratio    = 4
```

2.5.2.3 sys_config 配置项说明

下面将按照不同模块来解析 sys_config 中各个模块的配置含义。

- target 属性配置

在此配置下，与 PMU 相关的主要是 power_mode。power mode 这个节点就是为了告诉平台当前使用的是哪个 PMU 的哪种方案（有时在同一 SOC 平台同一 PMU 也可能出现电源树配置不一样的情况），在 boot0 阶段就会解析出该属性并调用调压接口进行调压。

```
power_mode <u32>
```

属性决定了当前SOC板型使用哪个PMU，需boot0代码支持解析，目前仅R818/MR813方案支持解析该节点。后续别的平台需支持时应根据boot0代码更新sysconfig中的版型选择说明。

```
0: axp81X,  
1: dummy  
2: axp806  
3: axp2202  
4: axp858
```

说明

AXP717 与其他 PMU 使用同一套代码框架，使用 AXP717 则选上3: axp2202即可。

• power_sply 属性配置

xxxx_vol <u32>

uboot阶段xxxx这路电是否开关及输出电压配置，其中xxxx为供电输出名。属性由前缀(100/000)和后缀组成。未配置的电在uboot阶段不会进行开关电和调压操作。

前缀：100，这路电在uboot阶段打开

前缀：110，这路电在uboot阶段打开，但是烧写时会关闭

前缀：000，这路电在uboot阶段关闭

后缀：3300，这路电输出电压设置为3300 mV

dcdcx_mode <u32>

uboot阶段强制将dcdcx设置为fpwm开关模式，提高这路抗负载扰动能力。该属性不配置默认为0。目前仅AXP806/AXP305/AXP81X/AXP803/AXP2202/AXP717支持该功能。

0: pfm-pwm模式自由切换

1: 强制pwm模式

battery_exist <u32>

强制电池存在状态，uboot阶段根据该属性决定是否做电池状态的相关判断。如果该属性不进行配置，默认为1。适用于部分无电持方案或factory_mode的无电池场景的调试

0: 强制认为无电池存在，uboot阶段不做电池相关状态判断

1: 认为电池存在，uboot阶段正常进行电池状态的相关判断

charge_mode <u32>

配置充电页面，根据该属性决定是否进入关机充电页面。

如果该属性不进行配置，默认为1。

适用于不需要充电页面，或者适配器唤醒直接开机的需求。

0: 不进入充电页面，适配器唤醒直接进入开机流程

1: 适配器唤醒进入充电流程

• power_delay 属性配置

xxxx_vol_delay <u32>

uboot阶段xxxx这路电调压后的延时时间，单位us。用于部分调压后需等电压稳定才能操作的模块。

如：twi，在上电时twi的电压不是3.3V，在uboot阶段需要将其升至3.3V，在电压抬升阶段是不能允许twi进行通信，不然就会产生错误。因此需要在调整电压的时候停止twi的功能。

该属性需与**power_sply**中的xxx_vol属性一块使用，当输出需要开关或调压时才需要进行延时。

• gpio_bias 属性配置

xx_bias <u32>

GPIOx口的耐压值设置，单位：mV。用于调整GPIOx口的耐压值，使其与GPIOx模块挂载的电压匹配，避免IO口损坏，提升信号质量。

xx_supply <char>

GPIOx模块挂载的输出电压名，名字格式需与**power_sply**中的xxx_vol一致。该属性如果配上，在GPIOx挂载的输出改编

后，会将对应的GPIOx bias耐压值修改过来。

- charger0 属性配置

pmu_safe_vol <u32>

电池安全电压，uboot里面会判断电池的电压值，电池电压值需要大于这个配置的电压值，才可以正常开机。配置示例：

pmu_safe_vol = 3400，代表电压需要大于3400mV才可以开机。

pmu_safe_ratio <u32>

电池安全电量，uboot里面会判断电池的电量百分比，电池电量需要大于这个配置的电量，才可以正常开机。配置示例：

pmu_safe_vol = 4，代表电量需要大于4%才可以开机。

2.5.3 uboot-board.dts 配置说明

在 Tina 系统的 Linux-5.4 内核版本及以后，取消了 sys_config.fex 配置，因此原先放在 sysconfig.fex 的配置迁移到 uboot-board.dts 中。在 Linux-4.9 中，部分平台也采用了这种方式。

📖 说明

如果方式配置目录下，同时存在 sys_config.fex 文件和 uboot-board.dts 文件。那么，Uboot 是只会调用到 uboot-board.dts 的。

2.5.3.1 AXP2101 uboot-board.dts 配置

该示例是以 R853 + AXP2101 作为例子的，运用在 Uboot-2018。其他采用 AXP2101 + Uboot-2018 的平台可以进行参考。

```
&power_sply {
    dcdc1_vol = <1003300>;
    dcdc2_vol = <1000900>;
    dcdc5_vol = <1200>;
    aldo1_vol = <1001800>;
    aldo2_vol = <1001800>;
    aldo3_vol = <1003300>;
    aldo4_vol = <1003300>;
    bldo1_vol = <1001800>;
    bldo2_vol = <1002800>;
    dldo1_vol = <1003300>;
    dldo2_vol = <1001200>;
    ntc_status = <1>;
};

&charger0 {
    pmu_safe_vol = <3400>;
    ntc_cur = <50>;
    safe_temp_H = <600>;
    safe_temp_L = <0xFFFFFFF6>;
    pmu_bat_temp_para1 = <4350>;
    pmu_bat_temp_para2 = <2650>;
    pmu_bat_temp_para3 = <2100>;
    pmu_bat_temp_para4 = <1650>;
    pmu_bat_temp_para5 = <1350>;
```

```
pmu_bat_temp_para6 = <1100>;
pmu_bat_temp_para7 = <850>;
pmu_bat_temp_para8 = <600>;
pmu_bat_temp_para9 = <456>;
pmu_bat_temp_para10 = <290>;
pmu_bat_temp_para11 = <245>;
pmu_bat_temp_para12 = <208>;
pmu_bat_temp_para13 = <175>;
pmu_bat_temp_para14 = <150>;
pmu_bat_temp_para15 = <111>;
pmu_bat_temp_para16 = <83>;
};

&gpio_bias { /* Avoid dtc compiling warnings. @TODO: Developer should modify this to the actual value */
    device_type = "gpio_bias";
    pc_bias = <1800>;
    pi_bias = <1800>;
};

&twi4 {
    clock-frequency = <200000>;
    pinctrl-0 = <&twi4_pins_a>;
    pinctrl-1 = <&twi4_pins_b>;
    no_suspend = <1>;
    twi_drv_used = <1>;
    status = "okay";
    pmu0: pmu@34 {
        compatible = "x-powers,axp2101";
        status = "okay";
        /* interrupts = <0 IRQ_TYPE_LEVEL_LOW>;
        * interrupt-parent = <&gic>; */
        x-powers,drive-vbus-en;

        wakeup-source;

        regulator0: regulators@0 {
            reg_dcdc1: dcdc1 {
                regulator-name = "axp2101-dcdc1";
            };
            reg_dcdc2: dcdc2 {
                regulator-name = "axp2101-dcdc2";
            };
            reg_dcdc3: dcdc3 {
                regulator-name = "axp2101-dcdc3";
            };
            reg_dcdc4: dcdc4 {
                regulator-name = "axp2101-dcdc4";
            };
            reg_dcdc5: dcdc5 {
                regulator-name = "axp2101-dcdc5";
            };
            reg_dc4ldo2: dc4ldo2 {
                regulator-name = "axp2101-dc4ldo2";
            };
            reg_dldo1: dldo1 {
                regulator-name = "axp2101-dldo1";
            };
            reg_dldo2: dldo2 {
                regulator-name = "axp2101-dldo2";
            };
        };
    };
};
```

```
reg_aldo1: aldo1 {
    regulator-name = "axp2101-aldo1";
};
reg_aldo2: aldo2 {
    regulator-name = "axp2101-aldo2";
};
reg_aldo3: aldo3 {
    regulator-name = "axp2101-aldo3";
};
reg_aldo4: aldo4 {
    regulator-name = "axp2101-aldo4";
};
reg_bldo1: bldo1 {
    regulator-name = "axp2101-bldo1";
};
reg_bldo2: bldo2 {
    regulator-name = "axp2101-bldo2";
};
};
```

2.5.3.2 AXP717 uboot-board.dts 配置

这里是采用 AXP717 + Uboot-2018 的平台的配置。

```
&power_sply {
    bldo1_vol = <1800>;
    bldo2_vol = <1001800>;
    cldo4_vol = <1103300>;
    dcdc1_mode = <1>;
    ext_dcdc0_mode = <1>;
    dcdc2_mode = <1>;
    bc12_mode = <0>;
    ntc_status = <2>;
};

&charger0 {
    pmu_safe_vol = <3400>;
    ntc_cur = <50>;
    safe_temp_H = <600>;//60
    safe_temp_L = <0xFFFFFCE>;//5
    pmu_bat_temp_para1 = <4592>;
    pmu_bat_temp_para2 = <2781>;
    pmu_bat_temp_para3 = <2125>;
    pmu_bat_temp_para4 = <1738>;
    pmu_bat_temp_para5 = <1390>;
    pmu_bat_temp_para6 = <1118>;
    pmu_bat_temp_para7 = <906>;
    pmu_bat_temp_para8 = <606>;
    pmu_bat_temp_para9 = <415>;
    pmu_bat_temp_para10 = <290>;
    pmu_bat_temp_para11 = <244>;
    pmu_bat_temp_para12 = <206>;
    pmu_bat_temp_para13 = <175>;
    pmu_bat_temp_para14 = <150>;
    pmu_bat_temp_para15 = <110>;
};
```

```
};
pmu_bat_temp_para16 = <83>;
};

&twi6 {
    clock-frequency = <200000>;
    pinctrl-0 = <&s_twi0_pins_a>;
    pinctrl-1 = <&s_twi0_pins_b>;
    twi-supply = <&reg_aldo3>;
    no_suspend = <1>;
    twi_drv_used = <1>;
    status = "okay";
    pmu0: pmu@34 {
        compatible = "x-powers,axp2202";
        status = "okay";
        /* interrupts = <0 IRQ_TYPE_LEVEL_LOW>;
        * interrupt-parent = <&gic>; */
        x-powers,drive-vbus-en;

        wakeup-source;

        regulator0: regulators@0 {
            reg_dcdc1: dcdc1 {
                regulator-name = "axp2202-dcdc1";
            };
            reg_dcdc2: dcdc2 {
                regulator-name = "axp2202-dcdc2";
            };
            reg_dcdc3: dcdc3 {
                regulator-name = "axp2202-dcdc3";
            };
            reg_rtcldo: rtcldo {
                /* RTC_LDO is a fixed, always-on regulator */
                regulator-name = "axp2202-rtcldo";
            };
            reg_aldo1: aldo1 {
                regulator-name = "axp2202-aldo1";
            };
            reg_aldo2: aldo2 {
                regulator-name = "axp2202-aldo2";
            };
            reg_aldo3: aldo3 {
                regulator-name = "axp2202-aldo3";
            };
            reg_aldo4: aldo4 {
                regulator-name = "axp2202-aldo4";
            };
            reg_bldo1: bldo1 {
                regulator-name = "axp2202-bldo1";
            };
            reg_bldo2: bldo2 {
                regulator-name = "axp2202-bldo2";
            };
            reg_bldo3: bldo3 {
                regulator-name = "axp2202-bldo3";
            };
            reg_bldo4: bldo4 {
                regulator-name = "axp2202-bldo4";
            };
            reg_cldo1: cldo1 {
                regulator-name = "axp2202-cldo1";
            };
        };
    };
};
```

```
};
reg_cldo2: cldo2 {
    regulator-name = "axp2202-cldo2";
};
reg_cldo3: cldo3 {
    regulator-name = "axp2202-cldo3";
};
reg_cldo4: cldo4 {
    regulator-name = "axp2202-cldo4";
};
reg_cpusldo: cpusldo {
    regulator-name = "axp2202-cpusldo";
};
};
};

&gpio_bias { /* Avoid dtc compiling warnings. @TODO: Developer should modify this to the actual value */
    device_type = "gpio_bias";
    pc_bias = <1800>;
    pl_supply = "aldo3_vol";
};

&power_delay {
    device_type = "power_delay";
    aldo3_vol_delay = <20000>;
};
```

2.5.3.3 uboot-board.dts 配置项说明

- power_sply 属性配置

xxxx_vol <u32>

uboot阶段xxxx这路电是否开关及输出电压配置，其中xxxx为供电输出名。属性由前缀(100/000)和后缀组成。未配置的电在uboot阶段不会进行开关电和调压操作。

前缀：100，这路电在uboot阶段打开

前缀：110，这路电在uboot阶段打开，但是烧写时会关闭

前缀：000，这路电在uboot阶段关闭

后缀：3300，这路电输出电压设置为3300 mV

ntc_status <0/1>

配置是否开启电池温度检测，如果配置为1，则在进入uboot时会检测电池的温度，若温度不在配置的范围，则会立马自动关机。反则，开机不会受到电池温度的影响。与配置safe_temp_H和safe_temp_L配合使用。

bc12_mode <0/1>

配置是否开启BC1.2功能，自动检测接入PC从而限流输入电流。

dcdcx_mode <0/1>

配置该路DCDC输出模式。0为自动选择；1为固定PWM模式。固定为PWM模式的话，可以使得电压波形更加稳定。

说明

与 sys_config.fex 功能一致，如果存在 sys_config.fex 的话，sys_config.fex 的配置会将 uboot.dts 的配置给覆盖，即优先级更高。

- charger0 属性配置

pmu_safe_vol <u32>

电池安全电压，uboot里面会判断电池的电压值，电池电压值需要大于这个配置的电压值，才可以正常开机。配置示例：
pmu_safe_vol = 3400，代表电压需要大于3400mV才可以开机。

ntc_cur <u32>

这个是测量电池温度时，芯片从测试温度引脚输出的电流。是用于适应不同的NTC电阻，可以配置成20uA/40uA/50uA/60uA，当NTC电阻选10Kohm类型时需要把电流设为50uA。

safe_temp_H <u32>

允许开机的最高安全电池温度，当电池温度高于设定的温度，则会立马关机。配置示例：safe_temp_H = <600>，这里表示电池温度允许最高60度。

safe_temp_L <u32>

允许开机的最低安全电池温度，当电池温度低于设定的温度，则会立马关机。配置示例：safe_temp_H = <0xFFFFFFF6>，这里表示电池温度允许最低-10度。

pmu_bat_temp_para <u32>

与board.dts的一致，直接复制boatd.dts中的配置到此处就可以使用。这里配置在uboot中加载电池温度参数，主要是为了能读取电池的温度。

- gpio_bias 属性配置

px_bias <u32>

GPIOx口的耐压值设置，单位：mV。用于调整GPIOx口的耐压值，使其与GPIOx模块挂载的电压匹配，避免IO口损坏，提升信号质量。

px_supply <string>

配置某一组GPIO口的供电regulator，使得在Uboot阶段，就打开对应的regulator，给对应的GPIO口供电。否则GPIO口无电的话，连接的外设也无法工作。

- power_delay 属性配置

xxxx_vol_delay <u32>

uboot阶段xxxx这路电调压后的延时时间，单位us。用于部分调压后需要等电压稳定才能操作的模块，如twi。该属性需要与power_sply中的xxxx_vol配合使用，当输出需要开关或者调压时，才需要用到本配置去进行延时。

- twi 属性配置

主要用于配置当前 pmu 有多少 regulator 以及对应的 regulator 名称，配置方法与 Device Tree 一致，可以参考章节《[Device Tree 配置说明](#)》。

2.5.4 kernel menuconfig 配置说明

在 Tina SDK 根目录运行 make kernel_menuconfig，进行内核配置修改，进入配置界面按以下步骤进行修改。

2.5.4.1 AXP2101 配置

- PMU 控制器

-> Device Drivers
-> Multifunction device drivers
<*> X-Powers AXP2101 PMICs with I2C

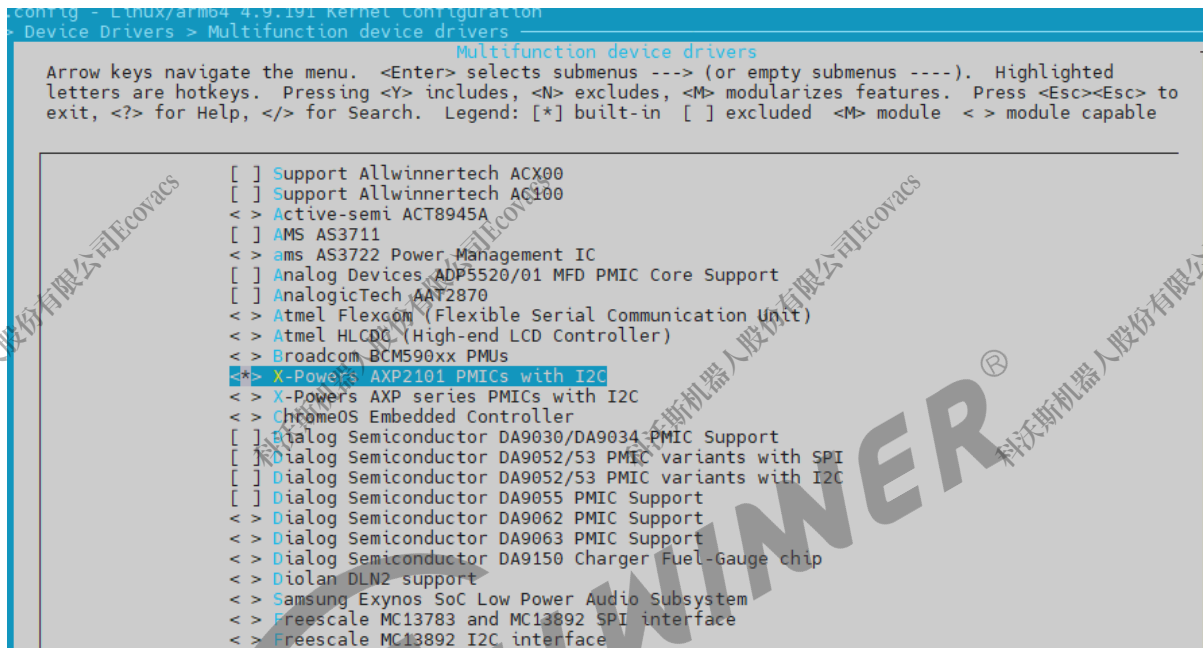


图 2-3: axp2101-pmu-control-config

- regulator

-> Device Drivers
-> Voltage and Current Regulator Support
<*> X-POWERS AXP2101 PMIC Regulators

```
Voltage and Current Regulator Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc> <Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable

--- Voltage and Current Regulator Support
[ ] Regulator debug support
<*> Fixed voltage regulator support
<*> Virtual regulator consumer support
<> Userspace regulator consumer support
<> Active-semi act8865 voltage regulator
<> Analog Devices AD5398/AD5821 regulators
<> Freescale i.MX on-chip ANATOP LDO regulators
<*> X-POWERS AXP2101 PMIC Regulators
<> Dialog Semiconductor DA9210 regulator
<> Dialog Semiconductor DA9211/DA9212/DA9213/DA9214/DA9215 regulator
<> Fairchild FAN53555 Regulator
<> GPIO regulator support
<> Intersil ISL9305 regulator
<> Intersil ISL6271A Power regulator
<> National Semiconductors LP3971 PMIC regulator driver
<> National Semiconductors LP3972 PMIC regulator driver
<> TI/National Semiconductor LP8720/LP8725 voltage regulators
<> TI LP8755 High Performance PMU driver
<> LTC3589 8-output voltage regulator
```

图 2-4: axp2101-regulatorot-config

- charger

```
-> Device Drivers
-> Power supply class support
<*> AXP2101 power supply driver - ext
```

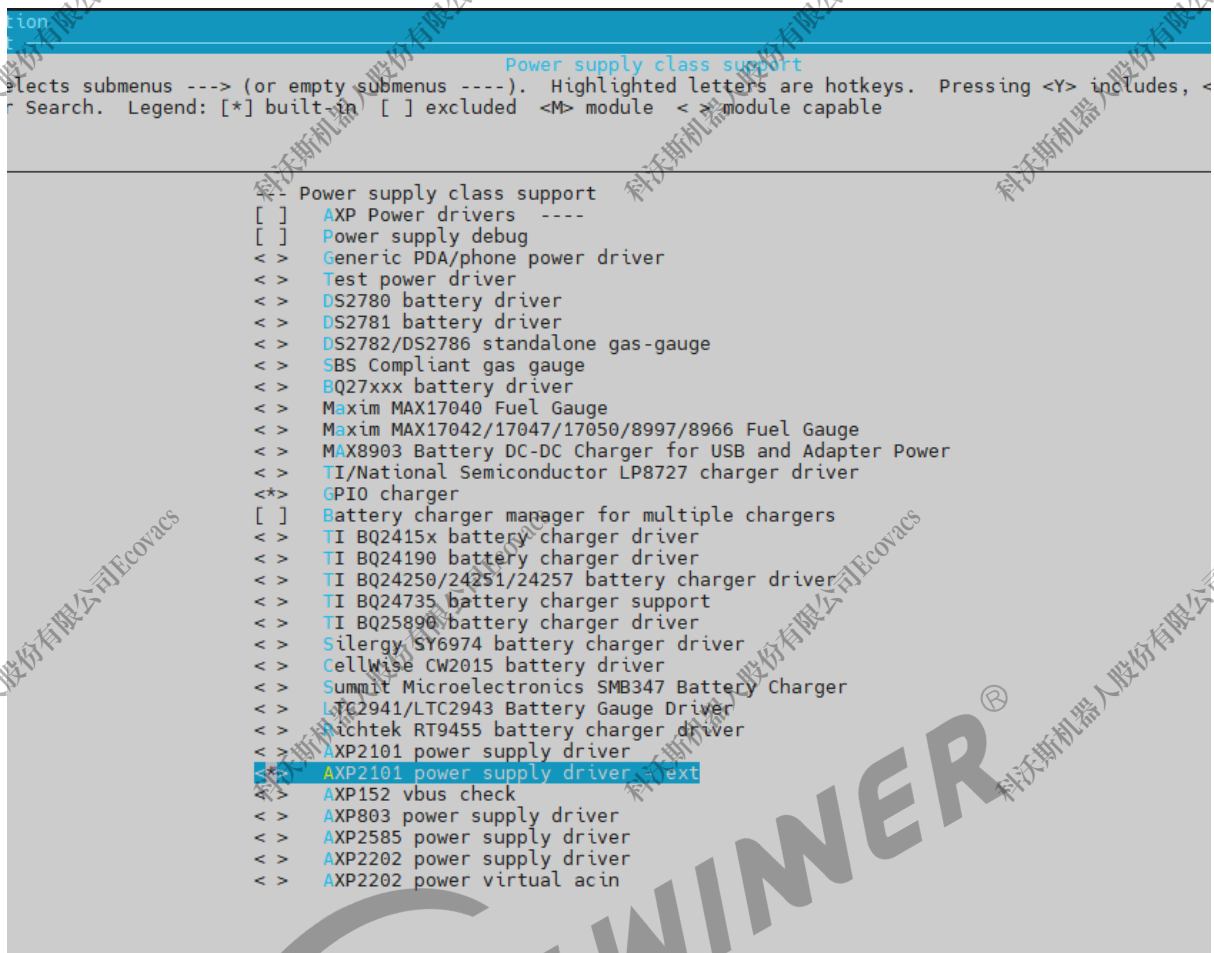


图 2-5: axp2101-charger-config

- power key

```
-> Device Drivers
  > Input device support
    -> Miscellaneous devices
      < * > X-Powers AXP2101 power button driver
```

```

Miscellaneous devices
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

--- Miscellaneous devices
<> Analog Devices AD714x Capacitance Touch Sensor
<> Atmel Capacitive Touch Button Driver
<> BMA150/SMB380 acceleration sensor support
<> NI Ettus Research USRP E3xx Button support.
<> MMA8450 - Freescale's 3-Axis, 8/12-bit Digital Accelerometer
<> MPU3050 Triaxial gyroscope sensor
<> Sharp GP2AP002A00F I2C Proximity/Opto sensor driver
<> Generic GPIO Beeper support
<> Polled GPIO tilt switch
<> Polled GPIO Decoder Input driver
<> ATI / Philips USB RF remote control
<> Keyspan DMR USB remote control
<> Kionix KXTJ9 tri-axis digital accelerometer
<> Griffin PowerMate and Contour Jog support
<> Yealink usb-p1k voip phone
<> C-Media CM109 USB I/O Controller
<> Regulator haptics support
[*] X-Powers AXP2101 power button driver
<> User level driver support
<> GPIO driver support
<> PCF8574 Keypad input device
<> PWM beeper support
<> Rotary encoders connected to GPIO pins
<> Analog Devices ADXL34x Three-Axis Digital Accelerometer
<> IMS Passenger Control Unit driver
<> VTI CMA3000 Tri-axis accelerometer
<> Windows-compatible SoC Button Array
<> TI DRV260X haptics support
<> TI DRV2665 haptics support
<> TI DRV2667 haptics support
<> stl3x1x Light driver support
<> hall_och175 hall driver support
<> LTR553ALS LIGHT SENSOR driver

```

图 2-6: axp2101-power-key-config

- virtual regulator

-> Device Drivers
 -> Voltage and Current Regulator Support
 [*] Virtual regulator consumer support

```

Voltage and Current Regulator Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

--- Voltage and Current Regulator Support
[ ] Regulator debug support
[*] Fixed voltage regulator support
[*] Virtual regulator consumer support
<> Userspace regulator consumer support
<> Active-semi act8865 voltage regulator
<> Analog Devices AD5398/AD5821 regulators
<> Freescale i.MX on-chip ANATOP LDO regulators
[*] X-POWERS AXP2101 PMIC Regulators
<> Dialog Semiconductor DA9210 regulator
<> Dialog Semiconductor DA9211/DA9212/DA9213/DA9214/DA9215 regulator
<> Fairchild FAN53555 Regulator
<> GPIO regulator support
<> Intersil ISL9305 regulator
<> Intersil ISL6271A Power regulator
<> National Semiconductors LP3971 PMIC regulator driver
<> National Semiconductors LP3972 PMIC regulator driver
<> TI/National Semiconductor LP8720/LP8725 voltage regulators
<> TI LP8755 High Performance PMU driver

```

图 2-7: axp2101-virtuaal-config

2.5.4.2 AXP717 配置

AXP717 与 AXP2101 公用同一份控制器、按键以及 regularotr 的代码，因此在 kernel menuconfig 里选的是 AXP2101 的选项。

- PMU 控制器

与 AXP2101 一致。

- regulator

与 AXP2101 一致。

- charger

```
-> Device Drivers
-> Power supply class support
<*> AXP2202 power supply driver
```

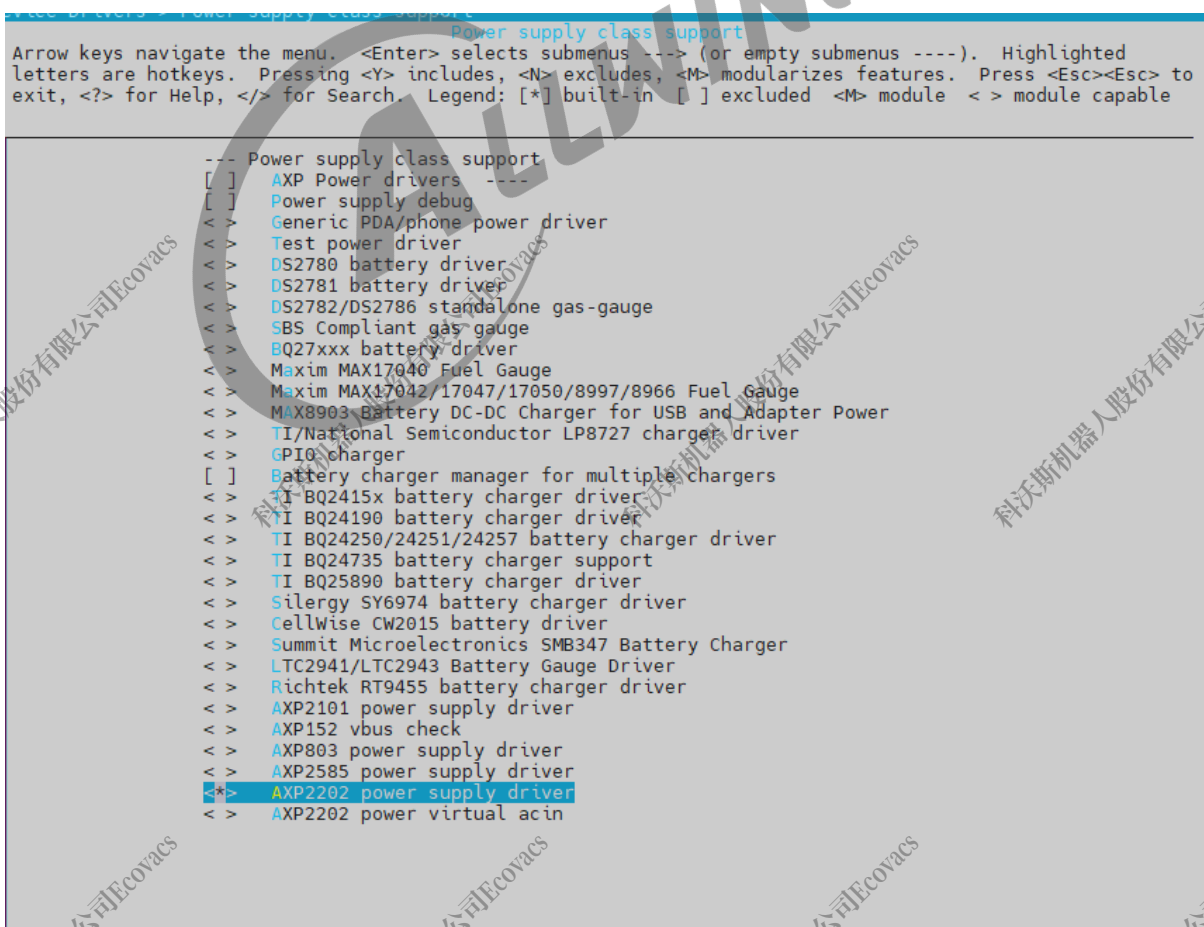


图 2-8: axp717-charger-config

- power key

与 AXP2101 一致。

- virtual regulator

与 AXP2101 一致。

- acin

```
-> Device Drivers
-> Power supply class support
  <> AXP2202 power virtual acin
```

Power supply class support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [] excluded <M> module <P> module capable

```
--- Power supply class support
[ ] AXP Power drivers ----
[ ] Power supply debug
< > Generic PDA/phone power driver
< > Test power driver
< > DS2780 battery driver
< > DS2781 battery driver
< > DS2782/DS2786 standalone gas-gauge
< > SBS Compliant gas gauge
< > BQ27xxx battery driver
< > Maxim MAX17040 Fuel Gauge
< > Maxim MAX17042/17047/17050/8997/8966 Fuel Gauge
< > MAX8903 Battery DC-DC Charger for USB and Adapter Power
< > TI/National Semiconductor LP8727 charger driver
< > GPIO charger
[ ] Battery charger manager for multiple chargers
< > TI BQ2415x battery charger driver
< > TI BQ24190 battery charger driver
< > TI BQ24250/24251/24257 battery charger driver
< > TI BQ24735 battery charger support
< > TI BQ25890 battery charger driver
< > Silergy SY6974 battery charger driver
< > Cellwise CW2016 battery driver
< > Summit Microelectronics SMB347 Battery Charger
< > LTC2941/LTC2943 Battery Gauge Driver
< > Richtek RT9455 battery charger driver
< > AXP2101 power supply driver
< > AXP152 vbus check
< > AXP803 power supply driver
< > AXP2585 power supply driver
< > AXP2202 power supply driver
[*] AXP2202 power virtual acin
```

图 2-9: axp717-acin-config

2.5.4.3 AXP717 配置 on Linux-5.15

说明

Linux-5.15 内核目前只有在 Tina V5.0 系统上使用到。

另外由于 Linux-5.15 的外设驱动采用了单独的 BSP 仓库，因此 kernel_menuconfig 的配置方法与上一个章节不一样。

- PMU 控制器

- > Allwinner BSP
- > Device Drivers
- > PMIC Drivers
 - <*> X-POWERS AXP2101 PMICs with I2Ca
 - <*> PMU_EXT PMICs with I2C

- regulator

- > Allwinner BSP
- > Device Drivers
- > PMIC Drivers
 - <*> X-POWERS AXP2101 PMIC Regulators
 - <*> PMU_EXT Power regulator

- charger

- > Allwinner BSP
- > Device Drivers
- > PMIC Drivers
 - <*> AXP2202 Power Supply Driver

- power key

- > Allwinner BSP
- > Device Drivers
- > PMIC Drivers
 - <*> X-POWERS AXP2101 Power Button Driver

上述四项配置全部都在一个页面内，配置图如下图所示：

```
linux/arm64 5.15.78 Kernel Configuration
r BSP > Device Drivers > PMIC Drivers
PMIC Drivers
keys navigate the menu. <Enter> selects submenus ---> (or empty submenu
s are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes t
<?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M>

[*] X-POWERS AXP2101 PMICs with I2C
[*] PMU_EXT PMICs with I2C
  *** Regulator Drivers ***
[*] X-POWERS AXP2101 PMIC Regulators
[*] PMU_EXT Power regulator
< > SUNXI PWM voltage regulator
  *** Powerkey Drivers ***
[*] X-POWERS AXP2101 Power Button Driver
  *** Power Supply Drivers ***
< > AXP803 Power Supply Driver
[*] AXP2202 Power Supply Driver
< > AXP2202 Power Virtual ACIN
< > AXP22X Power Supply Driver
```

图 2-10: axp717-config-on-linux-5.15

3 模块使用范例

3.1 外部 sysfs 节点

AXP 注册了许多外部 sysfs 节点可供调试使用。

3.1.1 Regulator

有关于 regulator 调试节点，主要调整各路电压输出的。

在路径 `/sys/class/regulator/` 下，有关于单路 regulator 属性的节点可以读取数值。

文件路径	功能	属性	设置值
<code>regulator.X/name</code>	各路输出名字	r	对应的输出的名字
<code>regulator.X/max_microvolts</code>	各路输出最大电压	r	对应的最大电压值，单位 uV
<code>regulator.X/min_microvolts</code>	各路输出最小电压	r	对应的最小电压值，单位 uV
<code>regulator.X/state</code>	各路输出状态	r	对应的输出状态， enabled/disabled: 开启\关闭
<code>regulator.X/num_users</code>	各路输出对应的设备个数	r	0\1\2\.....

因为不同 AXP 型号 regulator 对应的节点不一样，因此下面将分开不同 AXP 进行介绍。

- AXP717

电源名称对应表如下：

节点名字	原理图名称
<code>regulator.0</code>	dummy
<code>regulator.1</code>	usb0-vbus
<code>regulator.2</code>	usb1-vbus
<code>regulator.3</code>	dcdc1
<code>regulator.4</code>	dcdc2

节点名字	原理图名称
regulator.5	dcdc3
regulator.6	dcdc4
regulator.7	aldo1
regulator.8	aldo2
regulator.9	aldo3
regulator.10	aldo4
regulator.11	blldo1
regulator.12	blldo2
regulator.13	blldo3
regulator.14	blldo4
regulator.15	clldo1
regulator.16	clldo2
regulator.17	clldo3
regulator.18	clldo4
regulator.19	rtclldo
regulator.20	cpusldo
regulator.21	drivevbus

- AXP2101

节点名字	原理图名称
regulator.0	dummy
regulator.1	usb-vbus
regulator.2	dcdc1
regulator.3	dcdc2
regulator.4	dcdc3
regulator.5	dcdc4
regulator.6	dcdc5
regulator.7	rtclldo
regulator.8	rtclldo1
regulator.9	aldo1
regulator.10	aldo2
regulator.11	aldo3
regulator.12	aldo4
regulator.13	blldo1
regulator.14	blldo2
regulator.15	dlldo1
regulator.16	dlldo2
regulator.17	cpusldo

3.1.2 Virtual-consumer

在路径 `/sys/devices/platform/soc/7081400.s_twi/i2c-6/6-0034/` 下则有修改 regulator 输出电压的节点。

文件路径	功能	属性	设置值
<code>reg-virt-consumer.X/ max_microvolts</code>	设置输出最大电压	rw	对应的最大电压值，单位 uV
<code>reg-virt-consumer.X/ min_microvolts</code>	设置输出最小电压	rw	对应的最小电压值，单位 uV
<code>of_node/name</code>	各路输出名字	r	无

因为不同 AXP 型号 Virtual-consumer 对应的节点不一样，因此下面将分开不同 AXP 进行介绍。

- AXP717

virt-consumer 对应的电路的对应表如下：

节点名字	原理图名称
<code>reg-virt-consumer.1</code>	dcdc1
<code>reg-virt-consumer.2</code>	dcdc2
<code>reg-virt-consumer.3</code>	dcdc3
<code>reg-virt-consumer.4</code>	dcdc4
<code>reg-virt-consumer.5</code>	aldo1
<code>reg-virt-consumer.6</code>	aldo2
<code>reg-virt-consumer.7</code>	aldo3
<code>reg-virt-consumer.8</code>	aldo4
<code>reg-virt-consumer.9</code>	blldo1
<code>reg-virt-consumer.10</code>	blldo2
<code>reg-virt-consumer.11</code>	blldo3
<code>reg-virt-consumer.12</code>	blldo4
<code>reg-virt-consumer.13</code>	clldo1
<code>reg-virt-consumer.14</code>	clldo2
<code>reg-virt-consumer.15</code>	clldo3
<code>reg-virt-consumer.16</code>	clldo4
<code>reg-virt-consumer.17</code>	rtclldo
<code>reg-virt-consumer.18</code>	cpusldo

- AXP2101

virt-consumer 对应的电路的对应表如下：

节点名字	原理图名称
reg-virt-consumer.1	dcdc1
reg-virt-consumer.2	dcdc2
reg-virt-consumer.3	dcdc3
reg-virt-consumer.4	dcdc4
reg-virt-consumer.5	aldo1
reg-virt-consumer.8	aldo1
reg-virt-consumer.9	aldo2
reg-virt-consumer.10	aldo3
reg-virt-consumer.11	aldo4
reg-virt-consumer.12	blldo1
reg-virt-consumer.13	blldo2
reg-virt-consumer.14	dldo1
reg-virt-consumer.15	dldo2

3.1.3 Power_supply

在路径 `/sys/class/power_supply/` 下有 `power_supply` 相关的调试节点，可以读出电池以及供电当前状态和设置属性。

下表是电池相关的调试节点，路径为：`/sys/class/power_supply/*battery/`

文件路径	功能	属性	设置值
capacity	电池剩余电量	r	百分比，0\1\2\.....\100
capacity_alert_min	低电量警告阈值	r	百分比
capacity_level	当前充电等级	r	“UNKNOWN” 未知， “Critical” 接近没电，“LOW” 低电量，“NORMAL” 正常电 量，“HIGH” 高电量，“FULL” 满电
charge_counter	当前电池容量	r	单位 mAh
charge_full	充满电的电池容量	r	单位 mAh
constant_charge_current	恒定充电电流	r	单位 mA
energy_full_design	充满电的电池容量	r	单位 mAh
health	电池状况	r	“Unknown” 未知，“Good” 好，“Overheat” 过温，“Dead” 坏掉，“Over voltage” 过压， “Unspecified failure” 错误， “Cold” 冷
present	电池存在	r	0\1: 存在\不存在
serial_number	pmu 型号	r	string

文件路径	功能	属性	设置值
status	电池当前状态	r	“Unknown” 未知, “Charging” 正在充电, “Discharging” 放电, “Not charging” 未在充电, “Full” 满
temp	电池温度	r	单位 °C
temp_alert_min	电池高温预警阈值	r	单位 °C
time_to_empty_now	放电剩余时间	r	单位 min
time_to_full_now	充电剩余时间	r	单位 min
type	设备类别	r	“battery” 电池, “Mains” 火牛, “USB” USB
voltage_now	当前电压	r	单位 uA

下表则是供电相关的节点，路径为：/sys/class/power_supply/*usb

文件路径	功能	属性	设置值
input_current_limit	输入电流限流值	r	单位 mA
online	USB 是否在使用	r	0\1: 未使用\正在使用
present	USB 是否接上	r	0\1: 插上\没插上
serial_number	pmu 型号	r	string
type	设备类别	r	“battery” 电池, “Mains” 火牛, “USB” USB
voltage_min_design	DC 供电最小设计电 压	r	单位 uV
voltage_now	DC 供电时的电压大 小	r	单位 uV

说明

AXP717 没有使用 acin，因此接入 usb 或者适配器都是使用 usb-power-supply 的驱动，节点也是公用的。

3.2 Regulator 使用方法

3.2.1 内核代码调用 regulator 示例

以 DCDC1 为例，需要设置 DCDC1 最大输出电压值为 3.4V，需要设置目标电压值为 3V。

```
#include <linux/regulator/consumer.h>
```

```
struct regulator *regu= NULL;
```

```
int ret = 0;
regu= regulator_get(NULL, "axp2202_dcdc1");
if (IS_ERR(regu)) {
    pr_err("%s: some error happen, fail to get regulator\n", __func__);
    goto exit;
}

//set output voltage to 3V
ret = regulator_set_voltage(regu, 3000000, 3400000);
if (0 != ret) {
    pr_err("%s: some error happen, fail to set regulator voltage!\n", __func__);
    goto exit;
}

//enalbe regulator
ret = regulator_enable(regu);
if (0 != ret) {
    pr_err("%s: some error happen, fail to enable regulator!\n", __func__);
    goto exit;
}

//disalbe regulator
ret = regulator_disable(regu);
if (0 != ret) {
    pr_err("%s: some error happen, fail to disable regulator!\n", __func__);
    goto exit;
}

//put regulator, when module exit
regulator_put(regu);
```

💡 技巧

regulator_get 调用有两种方法：

第一种就是直接获取 dts 中 regulator 的句柄，比如说 axp2202_dcdc1。调用 regulator_get 时第一个参数不需要配置设备，直接写 NULL。

第二种是在 dts 的设备里配置 regulator 的节点，比如说 dts 某个设备里配置 regulator0 = "axp2202_dcdc1";，该设备调用时写法为：regulator_get(dev, "regulator0");。

3.2.2 Regulator shell 命令使用示例

AXP regulator 可以通过 shell 命令控制和设置其开关以及输出电压，各路文件节点建立在在 /sys/devices/platform 目录下。

可控的节点可参考上面 virtual 章节 [Virtual-consumer](#)。

以设置 DCDC1 输出最大电压为 3.3V，设置 AXP717 的目标电压为 3.0V 为例做说明。

```
cd /sys/devices/platform/soc/7081400.s_twi/i2c-6/6-0034/reg-virt-consumer.1
cat of_node/name //确认是否为DCDC1

//设置输出电压为3.0V
echo 3300000 > max_microvolts
echo 3000000 > min_microvolts

//关闭输出
```

```
echo 3300000 > max_microvolts  
echo 3000000 > min_microvolts  
echo 0 > min_microvolts
```

3.2.3 use_count 查看

根据上面 Regulator 章节 **Regulator** 找到对应的 regulator 节点，这里以 dcdc1 为例，其节点名称为 regulator.3，则在 /sys/class/regulator 目录下就有个 regulator.3 目录，regulator.3 目录有个 num_users 节点，cat 此节点就可以获得当前 use_count 值。

```
cat /sys/class/regulator/regulator.3/num_users
```

num_users 代表当前有多少设备使用了 regulator 节点用来控制输出电压。

4 电源管理应用

Tina 使用的电源管理应用为 healthd，主要实现的功能是：

1. 充电管理应用。在关机时，插入充电的 usb 线；或者是连接着充电 usb 线进行关机。两种情况下，不会使得系统直接开机，而是进入充电管理应用。屏幕上显示着充电的动画，显示一段时间后会息屏。如果短按按键的话，就会进行开屏/息屏的操作。如果长按按键的话，就会进行开机的操作。
2. 电源管理应用：在系统正常开机时，在后台跑的一个应用。主要用于检测电池或者是芯片的一个状态；如果电池电量低，就会发送低电量报警信息；如果发生了异常状态，比如说温度过高/过低，电压过低等等现象，就会使得系统进行关机起到保护的作用。

代码路径为：\${ROOT_DIR}/package/allwinner/healthd。

Tina healthd 代码结构：

```
src/BatteryMonitor.cpp    //获取电池状态的一些api实现
src/healthd.cpp           //healthd主函数
src/healthd_mode_charger.cpp //带动画显示的充电管理应用
src/healthd_mode_charger_without_animation.cpp //不带动画显示的充电管理应用
src/healthd_mode_tina.cpp  //系统运行中的电池管理应用
src/images                //存放充电时动画的文件
src/include               //头文件
files/healthd             //电池管理应用配置文件
files/healthd.init        //healthd自启动文件
libbattinfo/              //获取电池参数的一些api实现
demo/                    //一个使用广播获取healthd信息的应用
```

如何选上该应用，在 tina 根目录下，运行 make menuconfig，配置该软件包即可。软件包在 menuconfig 中的路径如下：

```
make menuconfig
-> Allwinner
-> healthd
-> [*]Broadcast battery information
-> [ ]Healthd Demo for test broadcast battery info function
-> [*]battery shutdown-charger
-> [ ]battery shutdown-charger with animation
```

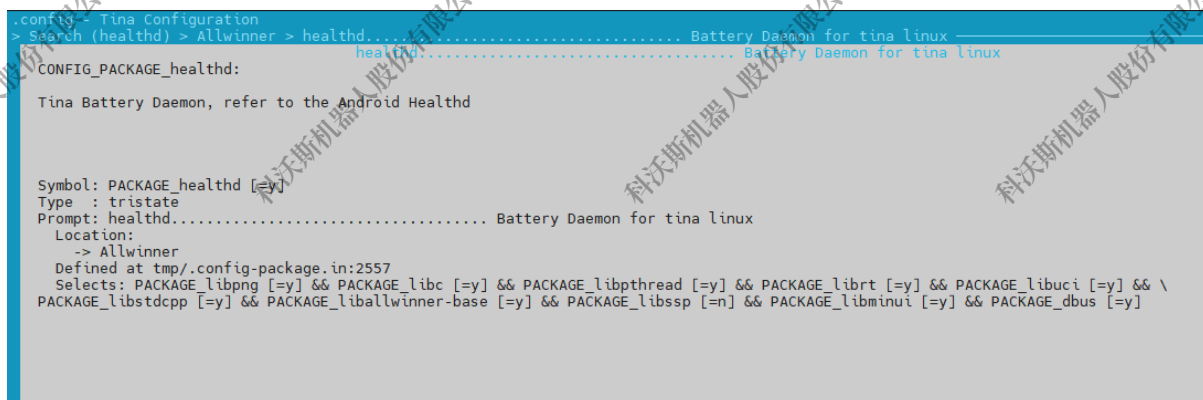


图 4-1: healthd_menuconfig1

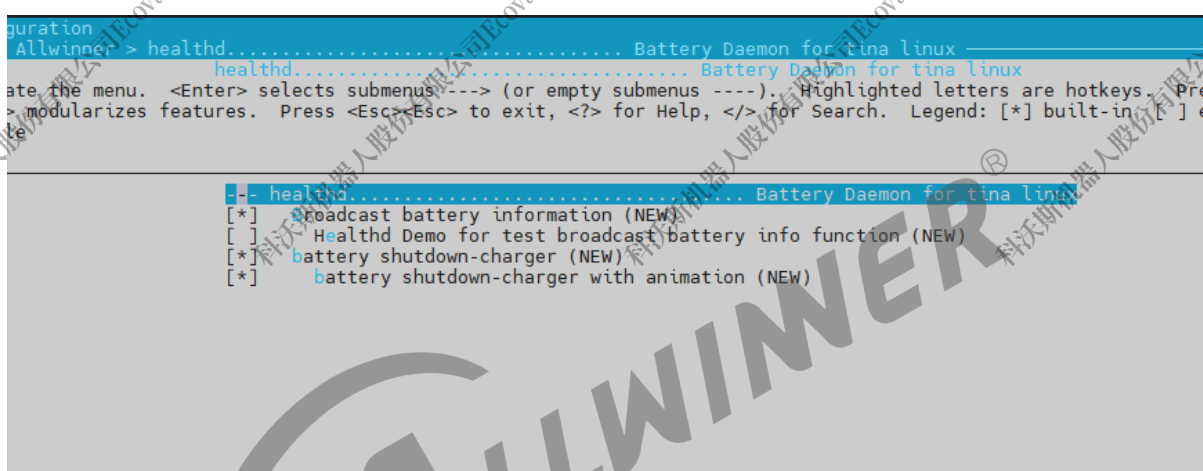


图 4-2: healthd_menuconfig2

4.1 healthd 机制

在加载完内核，启动系统时，由于 Makefile 将 healthd.init 放到了小机端的/etc/init.d/目录下，healthd 会自动加载，然后就会此次系统上电方式进行识别。如果判断上电方式为充电的话，则会启用充电管理应用 healthd_mode_charger，如果是其他方式的话，则启用普通的电池管理应用 healthd_mode_tina。

在 healthd 的 main 函数中，是通过结构体 healthd_mode_ops 去控制跑哪个应用的，代码中分别有两个已经初始化好的结构体可以对 healthd_mode_ops 进行赋值的，分别是：tina_ops，charger_ops。分别对应的功能是：普通的电池管理应用，充电管理应用。

```
468 int main(int argc, char **argv)
469 {
470     int ret;
471
472     /* tina_ops used to broadcast battery info */
473     healthd_mode_ops = &tina_ops;
474 #if 1
475     if(charger_ops.init) {
476         process_kernel_cmdline();
477         if (is_charger)
478             healthd_mode_ops = &charger_ops;
479     }
480 #else
481     healthd_mode_ops = &charger_ops;
482 #endif
483     ret = healthd_init();
484     if (ret) {
485         TLOGE("Initialization failed, exiting\n");
486         exit(2);
487     }
488
489     healthd_mainloop();
490     TLOGE("Main loop terminated, exiting\n");
491     return 0;
492 }
```

图 4-3: healthd_mode_ops

识别上电方式是通过 cmdline 中的一个参数 bootreason，bootreason 是从 uboot 中识别完成后传递过来的。bootreason 总共有以下几种方式：

1. 按键长按上电。bootreason 为 button。
2. IRQ 引脚拉低一段时间。bootreason 为 irq_low。
3. 关机时插入 usb 充电线。带电池的情况下 bootreason 为 charge，不带电池 bootreason 为 usb。
4. 充电时电池电压从 3.3V 以下上升至 3.3V。bootreason 为 charge。
5. 电池插入且电池电量健康。bootreason 为 battery。

当 bootreason 为 charge 时，会将变量 is_charger 置 1，然后才会进入充电管理应用。

```
186 static void import_kernel_nv(char *name, int for_emulator)
187 {
188     char *value = strchr(name, '=');
189     int name_len = strlen(name);
190
191     if (value == 0)
192         return;
193     *value++ = 0;
194     if (name_len == 0)
195         return;
196
197     if (!strncmp(name, "bootreason", 10) && name_len > 10) {
198         const char *boot_prop_name = name + 11;
199         DLOG("boot_prop_name=%s\n", boot_prop_name);
200
201         if (!strcmp(boot_prop_name, "charger"))
202             is_charger = 1;
203     }
204 }
```

图 4-4: is_charger

如果是进入充电管理应用的话，就不用启动系统正常的一些应用了。相关的实现代码为：

- 1.procd 启动方式：\${ROOT_DIR}/package/system/procd/patches/002-add_charger_mode_support.patch。
- 2.busybox 启动方式：\${ROOT_DIR}/package/busybox-init-base-files/files/etc/init.d/rcS 的 rc_charge_mode 函数。

4.2 充电管理应用

充电管理应用代码为：healthd_mode_charger.cpp

在进入 healthd.cpp 的 main 函数，等待资源初始化好之后，就会进入 mainloop 函数。mainloop 中会有一个 while (1) 的循环，循环会通过 epoll 机制阻塞住，等待内核 uevent 事件上报了，就会调用到 heartbeat 函数。

```
582 void healthd_mode_charger_heartbeat()  
583 {  
584     struct charger *charger = &charger_state;  
585     int64_t now = curr_time_ms();  
586     int ret;  
587  
588     DLOG("healthd_mode_charger_heartbeat\n");  
589     handle_input_state(charger, now);  
590     handle_power_supply_state(charger, now);  
591  
592     /* do screen update last in case any of the above want to start  
593      * screen transitions (animations, etc)  
594      */  
595     update_screen_state(charger, now);  
596 }  
597
```

图 4-5: heartbeat

AXP 驱动内部注册了一些 uevent 事件，比如说按键的 input 事件，或者是 power_supply 事件等。当有按键按下，或者电池、电源状态发生变化时，就会上报 uevent 事件。®

充电管理应用中的 heartbeat 函数：

1. 首先会检测按键的状态，查看按键是否按下，按下的时间多长，然后所根据不同按键状态做不同的处理。

```
543 static void handle_input_state(struct charger *charger, int64_t now)  
544 {  
545     process_key(charger, KEY_POWER, now);  
546     DLOG("handle_input_state\n");  
547     if (charger->next_key_check != -1 && now > charger->next_key_check)  
548         charger->next_key_check = -1;  
549 }
```

图 4-6: process_key

2. 然后再检测 power_supply 电源和电池相关的状态，检查是否还在充电，电池是否存在等等状态。

```

551 static void handle_power_supply_state(struct charger *charger, int64_t now)
552 {
553     DLOG("handle_power_supply_state charger->charger_connected=%d\n", charger->charger_connected);
554     if (!charger->have_battery_state)
555         return;
556     if (!charger->charger_connected) {
557         DLOG("handle_power_supply_state request_suspend(false)\n");
558         kick_animation(charger->batt_anim);
559         request_suspend(false);
560         if (charger->next_pwr_check == -1) {
561             charger->next_pwr_check = now + UNPLUGGED_SHUTDOWN_TIME;
562             DLOG("[%s PRId64 "] device unplugged: shutting down in %s PRId64 " (@ %s PRId64 ")\n",
563                 now, (int64_t) UNPLUGGED_SHUTDOWN_TIME, charger->next_pwr_check);
564         } else if (now >= charger->next_pwr_check) {
565             DLOG("[%s PRId64 "] shutting down\n", now);
566             reboot(RB_POWER_OFF);
567         } else {
568             /* otherwise we already have a shutdown timer scheduled */
569         }
570     } else {
571         /* online supply present, reset shutdown timer if set */
572         if (charger->next_pwr_check != -1) {
573             DLOG("[%s PRId64 "] device plugged in: shutdown cancelled\n", now);
574             kick_animation(charger->batt_anim);
575         }
576         charger->next_pwr_check = -1;
577     }
578 }
579 }
580 }

```

图 4-7: power_supply_state

3. 最后，根据先前两种状态，执行屏幕显示的功能。

```

339 static void update_screen_state(struct charger *charger, int64_t now)
340 {
341     struct animation *batt_anim = charger->batt_anim;
342     int cur_frame;
343     int disp_time;
344     if (!batt_anim->run || now < charger->next_screen_transition)
345         return;
346     if (!minui_init) {
347         if (healthd_config && healthd_config->screen_on) {
348             if (!healthd_config->screen_on(batt_prop)) {
349                 DLOG("[%s PRId64 "] leave screen off\n", now);
350                 batt_anim->run = false;
351                 charger->next_screen_transition = -1;
352                 if (charger->charger_connected)
353                     request_suspend(true);
354                 return;
355             }
356         }
357     }
358     gr_init();
359     gr_font_size(gr_sys_font(), &char_width, &char_height);
360     #ifndef CHARGER_DISABLE_INIT_BLANK
361     gr_fb_blank(true);
362     #endif
363     minui_init = true;
364     /* animation is over, blank screen and leave */
365     if (batt_anim->cur_cycle == batt_anim->num_cycles) {
366         reset_animation(batt_anim);
367         charger->next_screen_transition = -1;
368         gr_fb_blank(true);
369         DLOG("[%s PRId64 "] animation done\n", now);
370         if (charger->charger_connected) {
371             DLOG("[%s PRId64 "] request_suspend(true)---update_screen_state\n", now);
372             request_suspend(true);
373         }
374         return;
375     }
376 }
377 }
378 }
379 }
380 }
381 }

```

图 4-8: update_screen_state

屏幕显示功能用到的是 libminiui 的功能，此代码的路径为：\${ROOT_DIR}/package/allwinner/libminiui，这里就不对该代码展开阐述了。

整个充电管理应用的流程简单可如下方流程图所示。

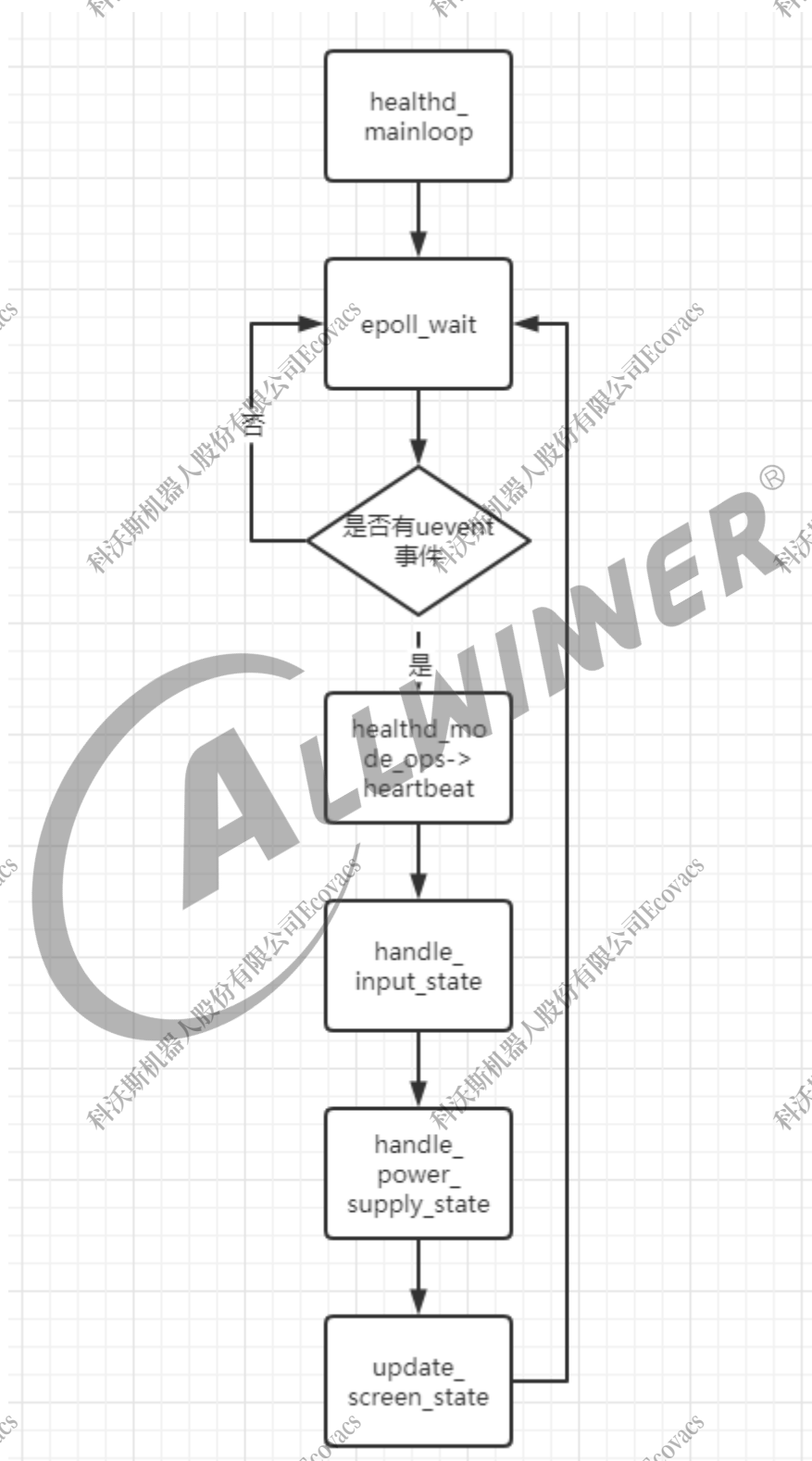


图 4-9: charger_flow_chart

4.3 电源管理应用

电源管理应用的代码为：healthd_mode_tina.cpp。

电源管理应用不同于充电管理应用，前者是不经过 heartbeat 去处理信息，而是直接通过获取 uevent 上报的事件，然后触发 uevent 的 event 处理函数 uevent_event()。函数中继而调用 healthd_battery_update() >> gBatteryMonitor->update() >> BatteryMonitor::update()。

在 BatteryMonitor::update() 函数中，会获取到电池的各种信息，收集完成后最终调用 healthd_mode_ops->battery_update() >> healthd_mode_tina_battery_update() 根据参数做不同的处理。

电源管理应用的大致流程可以如下流程图所示。

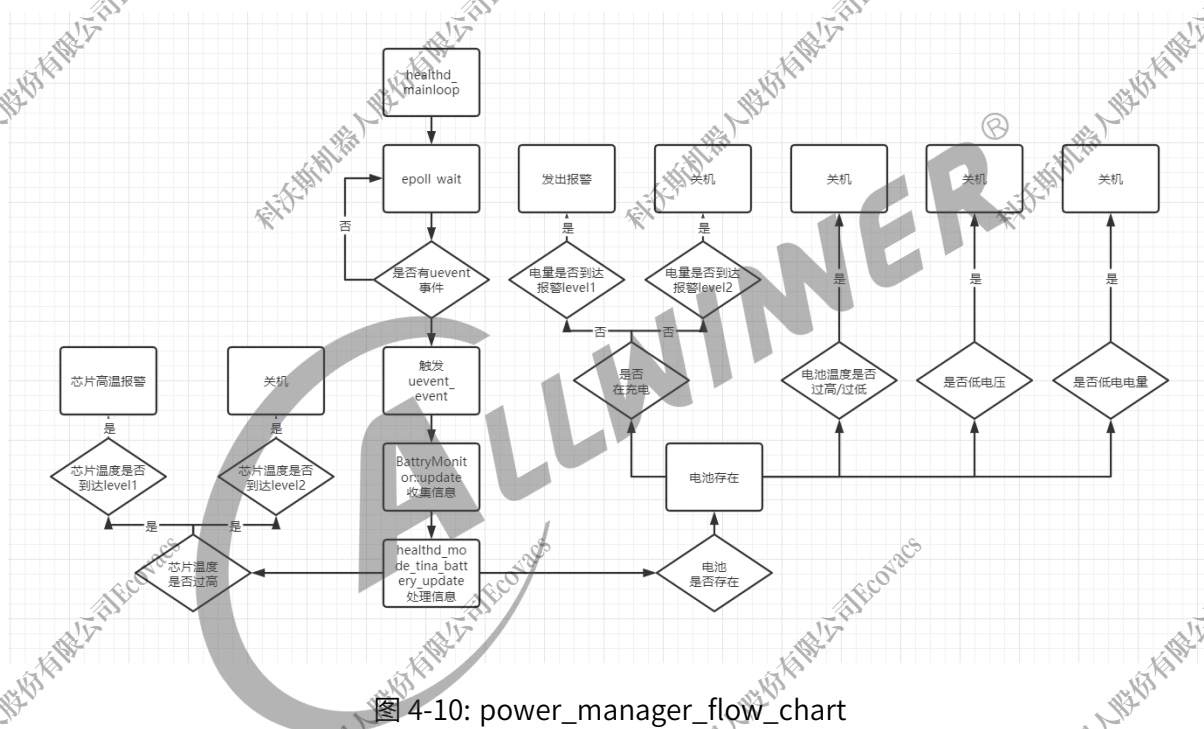


图 4-10: power_manager_flow_chart

下面将分别讲述电源管理应用中，有什么管理功能。

4.3.1 低电量警告

判断电池是否存在，且在没有充电的情况下，判断电池电量是否掉落到了设置的界限以下。一共有两个 level 的警报，默认设置为 15% 和 5%。

如果掉落到了 15% 了，那就发生了 level1 的警报，如果掉落到了 5%，就发生 level2 的报警。如下方代码所示。

```
238 /*low capacity handle */
239 if (batinfo_get_bat_capacity_level() == 1 &&
240     props->batteryLowCapWarn == false) {
241     TLOGE("Battery level is lower than warning level1\n");
242 }
243
244 if (batinfo_get_bat_capacity_level() == 2) {
245     TLOGE("Battery level is lower than warning level2\n");
246     // perform_shutdown(props);
247 }
248
249 low_capacity_warning = batinfo_get_bat_capacity_alert_level1();
250 if (low_capacity_warning >= 0 &&
251     props->batteryLevel <= low_capacity_warning) {
252     props->batteryLowCapWarn = true;
253 } else if (low_capacity_warning >= 0 &&
254     props->batteryLevel > low_capacity_warning) {
255     props->batteryLowCapWarn = false;
256 }
```

图 4-11: low_capacity_warning

4.3.2 低电压关机

判断电池是否存在，且在无充电的情况下，判断电池电压是否掉落到了设置的界限以下。电池电压界限默认为 3.4V。可通过文件 ./healthd/files/healthd 中的 bat_vol_low 去设置，单位为 uV。

如果电池电压掉落到设置的电压以下，就会调用 perform_shutdown(props) 进行关机。

```
258 /* low voltage handle */
259 if (props->batteryVoltage > 0 &&
260     get_healthd_config("bat_vol_low", &bat_vol_low) &&
261     props->batteryVoltage <= bat_vol_low) {
262     TLOGE("Battery Voltage:%d is lower than %duV\n",
263         props->batteryVoltage, bat_vol_low);
264     perform_shutdown(props);
265 }
266
```

图 4-12: low_voltage_shutdown

4.3.3 低电量关机

判断电池是否存在，且在无充电的情况下，判断电池电量是否掉落到了设置的界限以下。电池电量界限默认为 4%。可通过文件 ./healthd/files/healthd 中的 bat_cap_low 去自定义设置。

如果电池电量掉落到设置的电量以下，就会调用 perform_shutdown(props) 进行关机。

```
266
267     /* low capacity handle */
268     if (props->batteryLevel >= 0 &&
269         get_healthd_config("bat_cap_low", &bat_cap_low) &&
270         props->batteryLevel <= bat_cap_low) {
271         TLOGE("Battery Capacity:%d is lower than %d\n",
272             props->batteryLevel, bat_cap_low);
273         perform_shutdown(props);
274     }
```

图 4-13: low_capacity_shutdown

4.3.4 电池过高/过低温关机保护

如果有电池存在，且不管是否在充电，都会判断电池的 NTC 温度，如果温度变得过高/过低，都会导致程序执行异常温度关机。默认高温界限为 60 度，低温界限为-10 度。可通过文件./healthd/files/healthd中的 bat_temp_over 和 bat_temp_low 两个变量去自定义高低温界限。

```
278     if (props->batteryPresent) {
279         /* bat temp low & over handle */
280         if (props->batteryTemperature != -0xFF &&
281             get_healthd_config("bat_temp_low", &bat_temp_low) &&
282             props->batteryTemperature <= (bat_temp_low * 10)) {
283             TLOGE("props->batteryTemperature:%d <= bat_temp_low:%d\n",
284                 props->batteryTemperature, (bat_temp_low * 10));
285             perform_shutdown(props);
286         }
287         if (props->batteryTemperature != -0xFF &&
288             get_healthd_config("bat_temp_over", &bat_temp_over) &&
289             props->batteryTemperature >= (bat_temp_over * 10)) {
290             TLOGE("props->batteryTemperature:%d >= bat_temp_over:%d\n",
291                 props->batteryTemperature, (bat_temp_over * 10));
292             perform_shutdown(props);
293         }
294     }
295 }
```

图 4-14: over_ntc_shutdown

4.3.5 芯片高温警报和关机保护

不管有没有电池和充电的存在，每次都会判断的条件，就是芯片是否存在过高温的情况。如果有的话，温度达到了警报界限，则会发出警报（代码中默认是打印警报信息）；温度达到了关机界限，则会进行关机保护。默认的警报界限为 105 度，默认的关机界限为 120 度。可通过文件./healthd/files/healthd中 ic_high_temp_warning 和 ic_high_temp_shutdown 去自定义数值。

```
296  /*high ic temp handle */
297  props->batteryOverTempWarn = false;
298  ic_temp = batinfo_get_ic_temp();
299  if (ic_temp >= 0) {
300      if (get_healthd_config("ic_high_temp_shutdown", &ic_high_temp_shutdown)
301          && ic_temp >= ic_high_temp_shutdown) {
302          /*high ic temp shutdown */
303          TLOGE("ic_temp:%d >= ic_high_temp_shutdown:%d, system shutdown\n",
304              ic_temp, ic_high_temp_shutdown);
305          perform_shutdown(props);
306      }
307      if (get_healthd_config("ic_high_temp_warning", &ic_high_temp_warning)
308          && ic_temp >= ic_high_temp_warning) {
309          /*high ic temp warning */
310          props->batteryOverTempWarn = true;
311      }
312  }
313  led_flash(LED_BLUE, props->batteryOverTempWarn ? "timer" : "none");
314
```

图 4-15: over_tdie_shutdown

5 调试工具

在设备进行开发过程中，难免需要对各路电源进行调试，控制电源各路电压等操作，内核中提供了对电源调试的方式。

5.1 power key 调试方式

在用户空间调用 `getevent` 命令，通过标准 input 系统上报的 input 事件，可以确认 power key 是否能正常工作，是否能正常上报 input 事件。

```
add device 2: /dev/input/event1
name: "axp2202-pek"
poll 4, returned 1
/dev/input/event1: 0001 0074 00000001
poll 4, returned 1
/dev/input/event1: 0000 0000 00000000
poll 4, returned 1
/dev/input/event1: 0001 0074 00000000
poll 4, returned 1
/dev/input/event1: 0000 0000 00000000
poll 4, returned 1
/dev/input/event1: 0001 0074 00000001
poll 4, returned 1
/dev/input/event1: 0000 0000 00000000
poll 4, returned 1
/dev/input/event1: 0001 0074 00000000
poll 4, returned 1
/dev/input/event1: 0000 0000 00000000
```

按下按钮为 1，弹起为 0。0074 为 power 事件。

5.2 调试节点

5.2.1 regulator_summary 节点

regulator_summary 节点的详细路径为：

```
/sys/kernel/debug/regulator/regulator_summary
```

通过 `cat` 的 shell 命令可查询 regulator 的状态。

kernel 提供调试节点供电源进行调试进行，我们可以通过 kernel 的调试节点获取各路电源的各个详细状态。以 AXP717 的设备举例，首先需要 mount debugfs 文件系统。

```
mount -t debugfs none /sys/kernel/debug
```

```
cat /sys/kernel/debug/regulator/regulator_summary
```

regulator	use	open	bypass	voltage	current	min	max
regulator-dummy	0	8	0	0mV	0mA	0mV	0mV
dmic				0mV	0mA		
uart2				0mV	0mA		
uart1				0mV	0mA		
twi3				0mV	0mA		
twi2				0mV	0mA		
twi1				0mV	0mA		
twi0				0mV	0mA		
twi6				0mV	0mA		
usb0-vbus	0	0	0	5000mV	0mA	5000mV	5000mV
usb1-vbus	1	2	0	5000mV	0mA	5000mV	5000mV
5200000.ohci1-controller				0mV	0mA		
5200000.ehci1-controller				0mV	0mA		
axp2202-dcdc1	0	2	0	900mV	0mA	500mV	1540mV
cpu0				900mV	900mA		
reg-virt-consumer.1				0mV	0mA		
axp2202-dcdc2	0	1	0	950mV	0mA	500mV	3400mV
reg-virt-consumer.2				0mV	0mA		
axp2202-dcdc3	0	1	0	1200mV	0mA	500mV	1840mV
reg-virt-consumer.3				0mV	0mA		
axp2202-dcdc4	0	1	0	1000mV	0mA	1000mV	3700mV
reg-virt-consumer.4				0mV	0mA		
axp2202-aldol	0	3	0	2800mV	0mA	500mV	3500mV
sensor1				2800mV	3300mA		
sensor0				2800mV	3300mA		
reg-virt-consumer.5				0mV	0mA		
axp2202-aldol2	1	4	0	1800mV	0mA	500mV	3500mV
sensor1				1800mV	3300mA		
sensor0				1800mV	3300mA		
sensor0				1800mV	3300mA		
reg-virt-consumer.6				0mV	0mA		
axp2202-aldol3	0	1	0	3300mV	0mA	500mV	3500mV
reg-virt-consumer.7				0mV	0mA		
axp2202-aldol4	0	2	0	1800mV	0mA	500mV	3500mV
codec				1800mV	1800mA		
reg-virt-consumer.8				0mV	0mA		
axp2202-blol1	2	3	0	3300mV	0mA	500mV	3500mV
soc@03000000:wlan@0				3300mV	3300mA		
soc@03000000:wlan@0				3300mV	3300mA		
reg-virt-consumer.9				0mV	0mA		
axp2202-blol2	0	1	0	2500mV	0mA	500mV	3500mV
reg-virt-consumer.10				0mV	0mA		
axp2202-blol3	1	1	0	2800mV	0mA	500mV	3500mV
reg-virt-consumer.11				0mV	0mA		
axp2202-blol4	0	3	0	1500mV	0mA	500mV	3500mV
sensor1				1500mV	1800mA		
sensor0				1200mV	1800mA		
reg-virt-consumer.12				0mV	0mA		
axp2202-clol1	3	3	0	1800mV	0mA	500mV	3500mV
codec				1800mV	1800mA		
1-0036				1800mV	1800mA		

reg-virt-consumer.13				0mV	0mV		
axp2202-cldo2	0	1	0	3300mV	0mA	500mV	3500mV
reg-virt-consumer.14				0mV	0mV		
axp2202-cldo3	3	4	0	3300mV	0mA	500mV	3500mV
1-0036				3300mV	3300mV		
sdc2				0mV	0mV		
uart0				0mV	0mV		
reg-virt-consumer.15				0mV	0mV		
axp2202-cldo4	0	1	0	3300mV	0mA	500mV	3500mV
reg-virt-consumer.16				0mV	0mV		
axp2202-rtcldo	0	1	0	1800mV	0mA	1800mV	1800mV
reg-virt-consumer.17				0mV	0mV		
axp2202-cpusldo	0	1	0	900mV	0mA	500mV	1400mV
reg-virt-consumer.18				0mV	0mV		
axp2202-drivebus	0	0	0	0mV	0mA	0mV	0mV

节点打印含义解析如下：

- 1.regulator：各个 ldo 名称以及有何模块引用了该 ldo。
- 2.use：有多少个模块已经使用了该 ldo。
- 3.open：有多少个模块申请了该 ldo。申请和上一条 use 是不一样，申请指的是软件中有绑定了该句柄，不一定会调用输出电压。
- 4.bypass：指的是是否将电路引出到其他旁边，若为 1，则不输出电压，为 0 则是正常工作状态。
- 5.voltage：当前设置的电压。
- 6.current：当前设置的电流。
- 7.min：设置可输出最小的电压。
- 8.max：设置可输出最大的电压。

5.2.2 regmap registers 节点

shell 命令读写寄存器。

寄存器调试是指直接对 PMIC 的寄存器进行读写操作，此操作应该对寄存器有了解的情况下进行操作，不正确的操作方式将会导致芯片烧毁。在终端中，对抛出的调试结点进行读写操作，即可对寄存器进行读写操作。无论是读还是写寄存器，都应该首先挂载 debugfs 文件系统。

由于 PMIC 是通过 regmap 进行读写操作，应该可以使用 regmap 的调试结点进行对 PMIC 的读写访问操作。regmap 的调试结点在 debugfs 文件系统下面，通过对 regmap 调试结点的操作可以对 PMIC 的寄存器进行读写访问操作。

• 写操作

寄存器调试挂载在 debugfs 文件系统。

```
mount -t debugfs none /sys/kernel/debug
```

```
echo ${reg} ${value} > /sys/kernel/debug/regmap/${dev-name}/registers
```

实例：

```
echo 0xff 0x01 > /sys/kernel/debug/regmap/4-0034/registers
```

写 0xff 寄存器值为 0x01

- 读操作

```
寄存器调试挂载在debugfs文件系统。
mount -t debugfs none /sys/kernel/debug
cat /sys/kernel/debug/regmap/${dev-name}/registers
```

实例：

```
cat /sys/kernel/debug/regmap/4-0034/registers
读取pmic所有寄存器
```

5.2.3 axp_reg 节点

另外，还支持 axp 驱动自定义节点 axp_reg 读写寄存器。但是这种用法是不推荐的，因为有标准 regmap 方式来读写寄存器，根本没必要用私有非标的方式。示例如下：

```
往axp寄存器0x0f写入值0x55：
echo 0f55 > /sys/class/axp/axp_reg
读出axp寄存器0x0f的值：
echo 0x0f > /sys/class/axp/axp_reg
cat /sys/class/axp/axp_reg
```

5.2.4 debug_mask 节点

axp 驱动自定义节点 debug_mask 打开和关闭调试信息。相关调试信息参考具体的 PMIC 驱动。示例如下：

```
系统打印等级设置为8：
echo 8 > /proc/sys/kernel/printk
打开所有axp调试信息：
echo 0xf > /sys/class/axp/debug_mask
关闭所有axp调试信息：
echo 0x0 > /sys/class/axp/debug_mask
```

调试信息一般如下：

```
[ 712.458412] ic_temp = 45
[ 712.461311] vbat = 3977
[ 712.464082] ibat = -779
[ 712.466280] healthd: battery l=96 v=3977 t=30.0 h=2 st=3 c=-779 fc=5066880 chg=
[ 712.475174] charge_ibat = 0
[ 712.478448] dis_ibat = 779
[ 712.481545] ocv = 4073
[ 712.484239] rest_vol = 96
[ 712.487182] rdc = 123
[ 712.489862] batt_max_cap = 5066
[ 712.493472] coulomb_counter = 4857
[ 712.497583] AXP803_COULOMB_CTL = 0xe0
[ 712.501803] ocv_percentage = 86
[ 712.505436] col_percentage = 96
[ 712.509061] bat_current_direction = 0
[ 712.513386] ext_valid = 0
```

6 FAQ

6.1 如何调整充电电流大小

参考上述章节 dts 配置项说明的章节[dts 配置项说明](#)。

修改 dts 中的 pmu_runtime_chgcur, pmu_suspend_chgcur 和 pmu_shutdown_chgcur 分别是修改系统运行时，休眠和关机时的充电电流大小。

⚠ 注意

需要注意的是，不同款 AXP 芯片可承受的最大电流不一样，因此支持的最大充电电流都不一样，需要根据 AXP 芯片的 datasheet 去设计。

6.2 如何在用户空间获取电池电量等 power_supply 参数

在用户空间中有一些节点在 /sys/class/power_supply/battery 下，比如说获取电池电量就可以 cat /sys/class/power_supply/battery/capacity。

详情其余节点可以参考：上述章节[Power_supply](#)。

6.3 如何查看是否已经在烧录时更新了电池曲线参数

使用 AXP2101 作为例子，查看名为 rom_sel 寄存器。在 AXP2101 上，rom_sel 的寄存器地址为 0xa2 的 bit4。如果该 bit 为 1，证明是已经更新了电池曲线参数。如果该 bit 为 0，则是没有更新。通过使用 i2ctools 或者 axp_reg 节点都可以获取 AXP2101 的寄存器数据。axp_reg 节点的使用可以参考章节[axp_reg 节点](#)。

6.4 修改开机时 pmu 限制条件

比如说，如果 pmu 或者 bmu 不达到设置的温度或者电压范围之内，就不允许系统进行开机。这里通过修改 uboot.dts 中的参数即可实现功能，配置方法参考章节[《uboot-board.dts 配置项说明》](#)

6.5 ldo 实际电压与 regulator_summary 的不符

regulator_summary 节点示意图可以参考章节 [regulator_summary](#) 节点。

实际测量中，发现某一路 ldo 或者 dc/dc 的电压值，与 cat regulator_summary 节点得出的 current 电压值不符。如实际测量的 dc/dc 是 1.1V，但是 cat 电压中显示的是 0V。

其实这是正常的现象，因为在 boot0 或者 uboot 中，也会修改 ldo 或者 dc/dc 的电压值。regulator_summary 是内核中生成的节点，生成该节点时并没有获取真实的电压值，而是默认为 0V。只要内核的模块不通过标准的 regulator 接口去申请 ldo，并且设置电压值。那么 regulator_summary 节点就会默认 ldo 是没有被申请，且没有被使能，电压值为 0V。

6.6 多个模块都请求同一个 regulator 并设置不同的电压，regulator 以哪个模块设置的为准

在 Linux 的 regulator 框架中，如果多个模块都请求同一个 regulator 并设置不同的电压，那么 regulator 将会使用最后一个设置的电压值。

regulator 框架的全局共享的，当多个模块都需要申请同一路时，需要注意避免互相干扰。

著作权声明

版权所有 © 2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明



（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。